

基于烟幕干扰弹投放过程的策略优化

摘要

烟幕干扰弹的完整投放策略，涉及多个过程，每个过程中的参数给定直接影响最终的有效遮蔽时长。本文基于物理运动轨迹，对不同策略与遮蔽情况展开机理分析，构建投弹策略参数优化模型，根据不同情形调整决策，以最大化对真目标的有效遮蔽时长。

针对问题一：为计算在特定“单机单次投弹”情形下的有效遮蔽时长。首先，构建三维坐标系，分别对各参数对象展开运动轨迹分析以生成坐标的变化方程。其次，引入**射影曲线**的概念，借助该曲线构造烟幕遮挡判断准则。同时，从解析几何的角度结合代数判定推导约束条件，借助方程判别式构造出目标函数进而完成模型构建。最后以**解析法**来主要求解模型并以**采样法**加以对比验证，得到该情形下的有效遮蔽时长为 **1.3808s**。此外，对模型展开稳定性验证，结果表明模型具有很好的**鲁棒性**，结果的可靠性较高。

针对问题二：为求解在普遍“单机单次投弹”情形下的最优飞行及投弹策略。首先，引入飞行角度这一关键参数，以调整模型一中的坐标变化方程。其次，以**最大化遮蔽时间函数**为目标构造投弹策略优化模型。最后，以“**遗传算法-单纯形法融合算法**”而非直接遍历搜索来求解模型，解得有效遮蔽时长为 **4.5128s**、飞行速度为 **77.8098m/s**、飞行角度为 178.28° 。此外，对各参数展开**灵敏度分析**，验证了模型和决策的合理性。

针对问题三：为求解在“单机多次投弹”情形下的最优投弹策略。首先，给出该情形下几何与时间的相关约束。其次，对云团重叠情况展开分析，更新遮蔽判断准则后完成建模。最后，论证**反演**思想的合理性并借此求解模型，得到有效遮挡时长为 **6.5517s**、飞行速度为 **114.2299m/s**、飞行角度为 179.51° 。此外，以**仿真模拟**的形式展开结果可靠性验证，结果表明确实存在云团“重叠”情况，证明本文策略的有效性。

针对问题四：为求解在“多机单次投弹”情形下的最优投弹策略。首先，更新调整运动轨迹方程及相关约束条件。其次，对烟幕云团相对位置及有效遮挡情况展开分类讨论，在决策机理分析的基础上构建模型。同时，在选定策略思想后将上文融合算法与“**蒙特卡洛采样**”相结合来求解，最终解得整体有效遮蔽时长为 **10.4172s**。

针对问题五：为求解在“多个无人机、多个导弹、多次投弹”情形下的最优投弹策略。首先，根据题目情形引入对应示性函数并调整约束条件以在模型四的基础上构造模型。同时，以提升**遮蔽利用率**的角度展开决策分析。其次，将最优决策与 **K-Means 聚类**相结合，在**轨迹反演**与融合优化算法的基础上求解模型，得到有效遮蔽时长为 **12.4828s**。

本文特色在于，对结果的可靠性检验较丰富，通过鲁棒性分析、灵敏度检验、仿真模拟等方式检验模型，有助于验证结果的可靠性与决策的合理性。同时，决策与算法的选择多样，通过反演推导等形式规避直接正向遍历搜索的思路，可大大提升计算效率。

关键字： 决策机理分析；射影曲线；反演；遗传-单纯形法融合算法；可靠性检验

一、问题的背景与重述

1.1 背景资料

一套科学的烟幕干扰弹投放策略与无人机飞行参数规划，能为被保护目标免受来袭导弹探测提供关键保障。同时，由于无人机投弹量限制和云团下降遮挡机制等约束条件，以及来袭导弹与多架无人机存在明确初始位置等客观情况限制，设计出最合理的无人机飞行轨迹与投弹策略以使得真目标的有效遮蔽时间最大化成为了本文的研究目标。

1.2 问题重述

在该问题中，不同无人机规划与导弹来袭情况所对应的有效遮蔽时长均有差异。针对以下不同模拟情况，建立模型以设计出使得有效遮蔽时间最大化的干扰弹投弹策略：

问题一：考虑 1 架无人机 FY1、1 枚来袭导弹 M1、干扰弹投弹 1 次的模拟情况，根据给定的无人机的飞行轨迹与具体干扰弹投弹策略约束，建立模型以求解在该模拟情况下干扰弹对导弹 M1 的有效遮蔽时长。

问题二：考虑 1 架无人机 FY1、1 枚来袭导弹 M1、干扰弹投弹 1 次的模拟情况。在问题一的基础上取消无人机飞行轨迹与投弹策略的具体限制，以最大化有效遮蔽时间为目标建立模型，求解此时无人机飞行轨迹与干扰弹投弹及起爆策略。

问题三：考虑 1 架无人机 FY1、1 枚来袭导弹 M1、干扰弹投弹 3 次的模拟情况。以最大化有效遮蔽时长为目标构建模型，求解包含干扰弹起爆点坐标等信息的投弹策略。

问题四：考虑 3 架无人机、1 枚来袭导弹 M1、无人机各投弹 1 次的情况。以最大化有效遮蔽时长为目标构建数学模型，求解包含干扰弹投放坐标等信息的具体投弹策略。

问题五：考虑 5 架无人机、3 枚来袭导弹、无人机各投弹至多 3 次的模拟情况。以最大化有效遮蔽时长为目标构建模型，求解包含无人机飞行速度等信息的具体投弹策略。

二、模型假设

1. 假设无人机只会在其初始高度平面内执行任务，不会斜向上飞或斜向下飞。
2. 假设无人机收到任务指派到出发这一系列动作都是瞬时完成的，不会占用时间。
3. 假设烟幕云团不会因阳光、雨水、风等环境天气影响其形状以及有效遮蔽时间。
4. 假设无人机投放烟幕干扰弹这一操作是瞬时完成的。
5. 假设烟幕干扰弹被引爆成为烟幕云团球体的过程是瞬时完成的。
6. 假设云团遮蔽效果在 20 秒的有效时长内保持恒定，不会随时间推移而逐渐减弱。

三、问题分析

针对问题一，需计算单架无人机投放单个干扰弹时的有效遮蔽时长。核心是判断遮蔽有效性，本问引入射影曲线概念，以射影曲线上的点与对应时刻导弹位置点的连线和烟幕云团的位置关系为判断依据。在明确各部分运动规律后，采用解析法结合变时间步长计算有效遮蔽时长，通过采样法验证计算结果，并开展结果的鲁棒性分析。

针对问题二，不再给定特定情况，而是要通过制定尽可能延长遮蔽时间的策略方案，来求解具体的飞行与投弹策略。本文主要针对单架无人机投放干扰弹的一般情形展开研究，依据确立的有效遮蔽判定规则，寻找能使遮蔽时间最大化的策略，并通过对运动规律方程与判断准则的微调来展开方案筛选。最终采用遗传算法与 Nelder–Mead 单纯形法相结合的混合优化策略进行求解，并对相关参数开展灵敏度分析。

针对问题三，其为问题二的“单机多弹投放”拓展情形，仍以最大化遮蔽时间为目标，求解具体的飞行与投弹策略。该情形会出现烟幕云团重叠或者组合的情况，关键在于通过遮蔽成功判定条件来求解具体的遮蔽时间。本文选择采用“先确定引爆位置反演投放策略、再正向搜索验证”的双向思路求解投放策略，并通过仿真对比检验结果。

针对问题四，需在“多机单次投弹”背景下求解具体的飞行与投弹策略。分析可知，该问题可视为三个“问题二情形”的组合，并需额外考虑烟幕云团可能存在的重叠或组合状态，且其遮挡判断条件与前几问基本一致，核心目标仍为最大化有效遮蔽时间，最终本文选择通过“多子群 + 蒙特卡洛 + NM 局部优化”的算法思想求解该模型。

针对问题五，属于“多无人机、多次投弹、多枚导弹”的情形，需求解各无人机对应的飞行与投弹策略。本文在模型四的基础上增加约束条件并展开策略机理分析以构建模型，并基于最佳策略的思想来设计算法求解模型，得到整体最优飞行与投弹策略。

四、符号说明

符号	说明	符号	说明
(x_{d0}, y_{d0}, z_{d0})	导弹初始点坐标	(x_d, y_d, z_d)	导弹坐标
(x_{w0}, y_{w0}, z_{w0})	无人机出发点坐标	(x_w, y_w, z_w)	无人机坐标
(x_{g0}, y_{g0}, z_{g0})	干扰弹投放点坐标	(x_g, y_g, z_g)	烟雾干扰弹坐标
(x_{b0}, y_{b0}, z_{b0})	引爆点坐标	v_b	云团下降速度
(x_b, y_b, z_b)	烟雾云团中心坐标	ρ_B, C, D	线段比例系数
$I_{[\Delta]}$	用于判定某时刻目标是否遮蔽成功	$I_{[t]}$	用于判定是否处于云团有效时间内
α	飞行角度	v_w	无人机飞行速度

注：其余符号将在下文中给出具体说明。

五、模型的建立与求解

5.1 问题一：有效遮蔽时长计算模型

本题要求在给定飞行与投弹策略下计算干扰弹的有效遮挡时长，传统最直观的思路是利用导弹与目标间连线投影是否被云团完全覆盖来判断遮挡情况，而在本文中根据目标分布与导弹轨迹可知，投影的面积与形状一定是变化的且在模型中难以量化的，故本文选择以代数推导的形式从解析几何的角度来构建模型，并在后序以搜索类算法求解。

5.1.1 坐标系构建及运动轨迹分析

■ 坐标系构建

选择以假目标作为三维坐标系 $O-XYZ$ 的坐标原点，水平面作为三维坐标系的 xy 平面来构建坐标系，进而将整个问题转换到一个坐标系中来处理，构建结果如下所示。

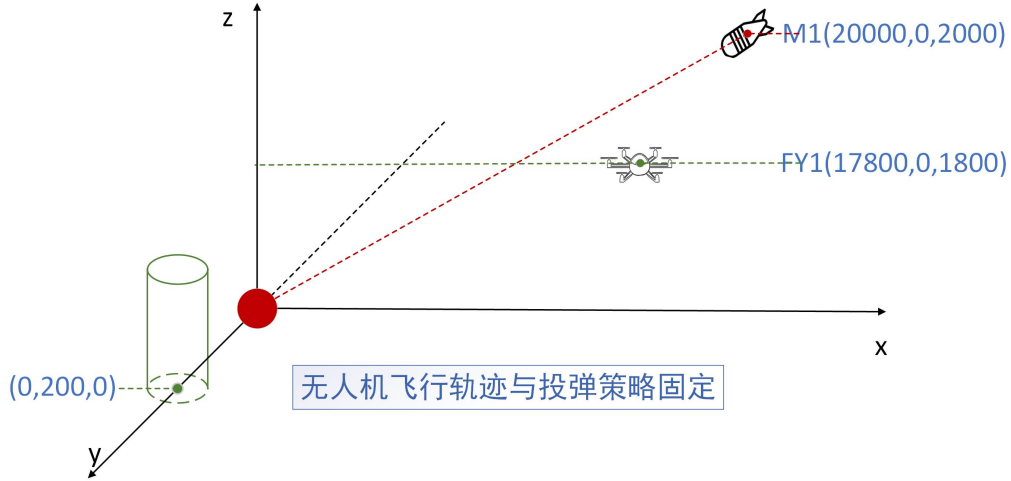


图 1 问题一坐标系构建

其中，红色虚线表示导弹 M1 的轨迹，绿色虚线表示无人机 FY1 的轨迹。

■ 导弹运动轨迹分析

已知无人机和来袭导弹在同一时刻出发，故本文选择设该时刻为初始时间点 $t_0 = 0$ 。同时由题已知，导弹采用直线路径射向假目标，即导弹轨迹为 xz 平面上的一条通过导弹初始点 (x_{d0}, y_{d0}, z_{d0}) 与原点的一条直线段。不妨设该直线段的斜率为固定的 k 值，即可得到导弹运动轨迹的轨迹方程以及导弹坐标 (x_d, y_d, z_d) 其具体的坐标变化方程如下：

$$\text{轨迹方程} = \begin{cases} z = kx \\ y = 0 \\ x \in [0, x_{d0}] \end{cases} \iff \text{坐标变化方程} = \begin{cases} z_d = z_{d0} - v_d t \sin \theta \\ x_d = x_{d0} - v_d t \cos \theta \\ y_d = 0 \\ x_d \in [0, x_{d0}] \end{cases} \quad (1)$$

■ 无人机运动轨迹分析

在问题一中，题目无人机设定的速度为 $120m/s$ ，且从出发点 (x_{w0}, y_{w0}, z_{w0}) 在 z_{w0} 的高空等高度匀速朝向假目标方向直线飞行。故知其轨迹为 xz 平面上一条水平的直线段，且可得到的无人机轨迹方程以及坐标 (x_{w0}, y_{w0}, z_{w0}) 其具体的坐标变化方程如下：

$$\text{轨迹方程} = \begin{cases} z = z_{w0} \\ y = 0 \\ x \in [0, x_{w0}] \end{cases} \iff \text{坐标变化方程} = \begin{cases} z_w = z_{w0} \\ x_w = x_{w0} - v_w t \\ y_w = 0 \\ x_w \in [0, x_{w0}] \end{cases} \quad (2)$$

■ 干扰弹运动轨迹分析

烟幕干扰弹在无人机出发后 t_g 时间被投放，由于无人机的运动，烟幕干扰弹在投放后会存在一个与无人机朝向相同的速度 v_w ，并且在竖直方向由于重力的作用会产生下个向下的加速度，因此整个烟幕干扰弹是在做一个**平抛运动**，运动轨迹是一段从被投点 (x_{g0}, y_{g0}, z_{g0}) 开始，在引爆点 (x_{b0}, y_{b0}, z_{b0}) 结束的抛物线，具体表达形式如下 [1]：

$$\text{轨迹方程} = \begin{cases} z = z_{g0} - \frac{1}{2}g \left(\frac{x_{g0} - x_b}{v_w} \right)^2 \\ y = 0 \\ x \in [x_{g0}, x_{b0}] \end{cases} \iff \text{坐标变化方程} = \begin{cases} z_g = z_{g0} - \frac{1}{2}g (t - t_{g0})^2 \\ x_g = x_{g0} - v_w (t - t_{g0}) \\ y_g = 0 \\ x_g \in [x_{g0}, x_{b0}] \end{cases} \quad (3)$$

同时，本文选择在此以图像的方式直观呈现本题无人机、干扰弹、云团的运动轨迹：

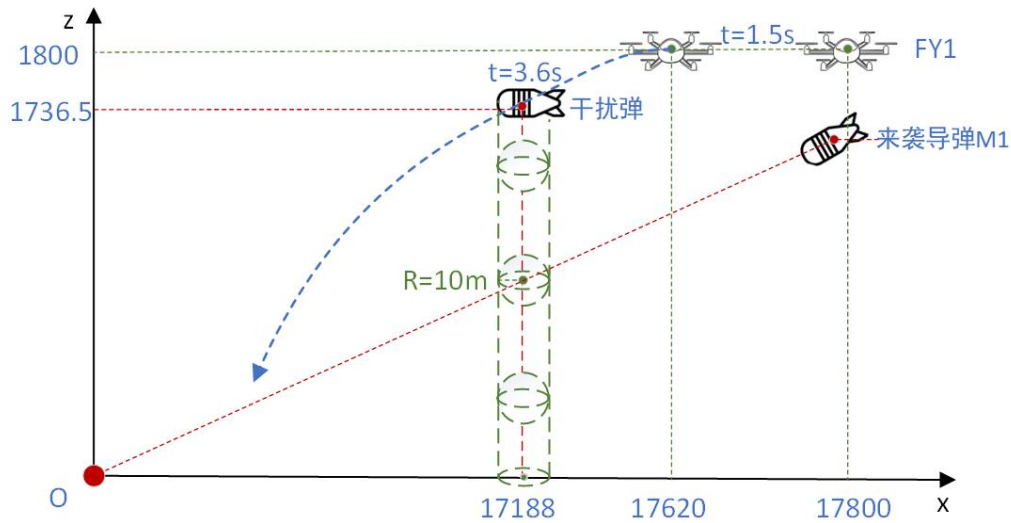


图2 运动轨迹 xz 剖面示意图

其中，无人机的投弹点坐标与干扰弹的起爆点坐标为基于题目数据直接计算所得。

■ 云团运动轨迹分析

烟幕干扰弹经过一段平抛运动后，在时间 t_b 被引爆成为烟幕云团，由本题背景资料可知，采用了特定技术的烟幕云团会从被引爆点 (x_{b0}, y_{b0}, z_{b0}) 开始以速度 $v_b = 3m/s$ 匀速下沉，故其轨迹是垂直于 x 轴的一条线段，轨迹方程与云团中心坐标变化方程如下：

$$\text{轨迹方程} = \begin{cases} z \in [0, z_{b0}] \\ y = 0 \\ x = x_{b0} \end{cases} \iff \text{坐标变化方程} = \begin{cases} z_b = z_{b0} - v_b (t - t_b) \\ x_b = x_{b0} \\ y_b = 0 \\ z_b \in [0, z_{b0}] \end{cases} \quad (4)$$

5.1.2 关于导弹与真目标射影曲线的几何分析及证明

首先，自定义**射影曲线**：射影曲线是由导弹视角所能看到的圆柱体的视野边界所构成的曲线。因此只要射影曲线不被发现就意味着该时刻整个真目标不会被导弹发现。

其次，分析射影曲线的意义在于可以实现从立体采点到面采点再到最终的曲线采点，实现从“体->面->线”的采点飞跃，以**减少计算时采点的工作量**，即只需要讨论射影曲线上的采点即可等价判断遮蔽成功与否，为模型求解减少计算量提供关键帮助 [2]。

同时，关于整个过程中射影曲线的分类情况，可具体分为两种情况：导弹垂直高度大于圆柱体、导弹垂直高度小于圆柱体，其各自对应的理论分析及可实现性论证如下。

■ 情况一：导弹垂直高度高于圆柱体高度

当导弹垂直高度高于目标圆柱体高度时，其射影边界如右图绿色阴影部分所示。可知射影边界由圆柱底部及两条直线段和一段椭圆弧所构成，图中已标注核心关键点的射影情况，故根据向量以及共线性可有：

$$\begin{cases} \overrightarrow{OC'} = \overrightarrow{OC} + \overrightarrow{CC'} = \overrightarrow{OC} + \rho_C \overrightarrow{PC} \\ \overrightarrow{OD'} = \overrightarrow{OD} + \overrightarrow{PD'} = \overrightarrow{OD} + \rho_D \overrightarrow{PD} \\ \overrightarrow{OB'} = \overrightarrow{OB} + \overrightarrow{PB'} = \overrightarrow{OB} + \rho_B \overrightarrow{PB} \end{cases}$$

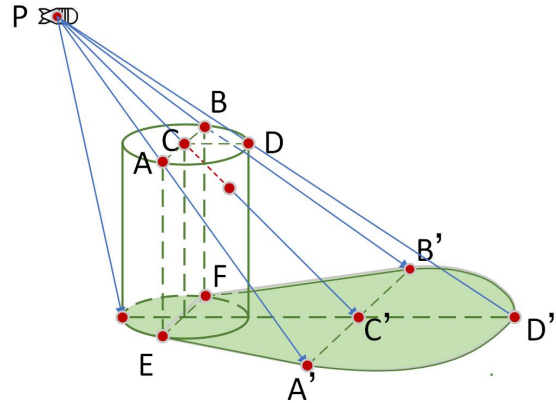


图3 射影曲线阴影图

其中， ρ_C 、 ρ_D 、 ρ_B 为待定参数，分别用于求解三个对应射影点 C' 、 D' 、 B' 的坐标。同时，由于三个对应射影点 C' 、 D' 、 B' 的纵坐标为零，可得具体数学表达式如下：

$$\rho_C = -\frac{z_C}{\overrightarrow{PC}_z}, \rho_D = -\frac{z_D}{\overrightarrow{PD}_z}, \rho_B = -\frac{z_B}{\overrightarrow{PB}_z} \quad (5)$$

将上述参数表达式代入原式，可得三个点的投影点坐标 (以 C 的投影点 C' 为例)：

$$\begin{cases} x_{C'} = x_C - \frac{\overrightarrow{PC}_x}{\overrightarrow{PC}_z} z_C \\ y_{C'} = y_C - \frac{\overrightarrow{PC}_y}{\overrightarrow{PC}_z} z_C \\ z_{C'} = 0 \end{cases} \quad (6)$$

其余两个射影点同理，在正文中不再赘述。由于 C' 、 D' 、 B' 为椭圆的端点，故根据计算的投影点坐标信息即可得到椭圆的曲线表达式。同时，基于 C' 、 B' 坐标与对称性可得 A' 坐标，进而得到两条射影曲线直线部分表达式。至此，已完成在该情况下的论证，即在导弹垂直高度高于圆柱体高度时，求解圆柱体真目标射影曲线的可实现性。

■ 情况二：导弹垂直高度低于圆柱体高度

根据导弹路径分析，可知还存在导弹垂直高度低于圆柱体高度的情况如图 4 左侧所示，此时过导弹位置点作关于圆柱体的两个切面，交线很显然是射影曲线的一部分，且上下底面的射影曲线情况分析类似。对于上底面，令 KP 线段与 JP 线段所呈夹角为 θ ，同理，令 KP 线段与 LP 线段所呈夹角为 β ，根据几何关系，易得三角形相似关系：

$$\triangle UJG \sim \triangle MGP \quad (7)$$

同时，根据图 4 右侧的截面图和射影几何关系及正切几何关系 $\tan \theta = \frac{GM}{MP}$ 可得：

$$\begin{cases} UG = R - R \cos \gamma \\ UJ = UG \cdot \tan \theta \end{cases} \quad (8)$$

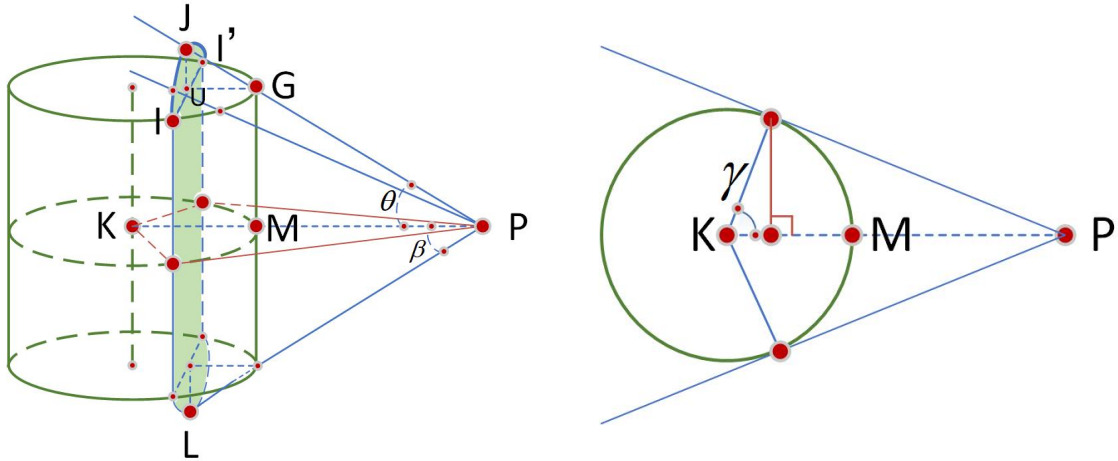


图 4 射影曲线阴影图及截面图

由此可计算出椭圆弧 IJI' 三点的具体坐标，从而得到射影曲线椭圆弧段的表达式，同理根据 β 夹角可获得下方椭圆弧的具体信息。至此，已完成在该情况下的论证，即在导弹垂直高度小于圆柱体高度时，求解圆柱体真目标射影曲线的可实现性。

5.1.3 烟幕遮挡判断

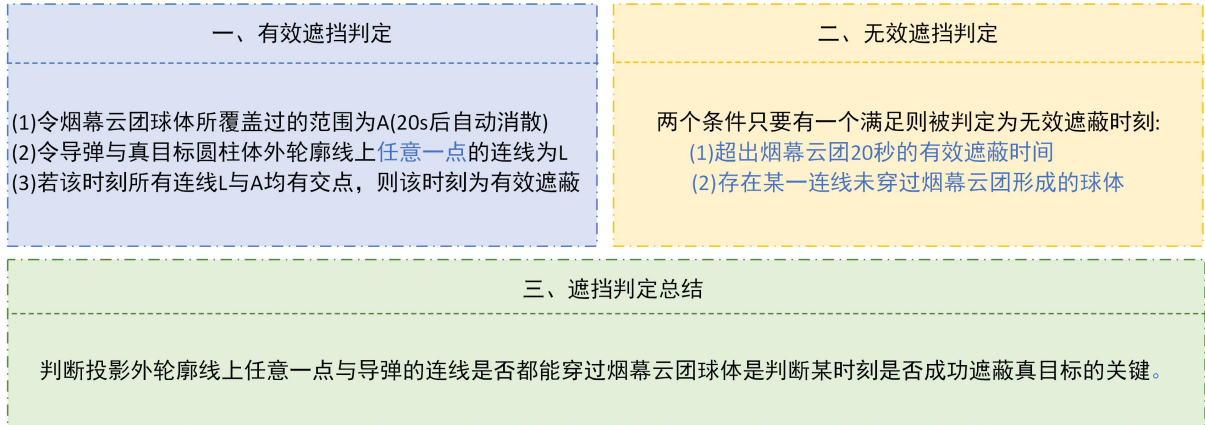


图5 判定核心逻辑规则

■ 几何关系准备

由于导弹与云团在整个过程属于动态过程，为便于分析，本文选择针对某个时刻的情况来进行具体的建模与分析。假定某一时刻，本文给定从导弹视角下圆柱的投影图形 P 的外轮廓线为 $L(x, y, z)$ ，此轮廓线所包含的范围即为无烟幕云团时导弹所能看到的真目标的范围。因此根据解析几何知识可得，可得到导弹与投影上点的连线方程为：

$$\frac{x - x_1}{x_d - x_1} = \frac{y - y_1}{y_d - y_1} = \frac{z - z_1}{z_d - z_1} \quad (9)$$

其中， (x_1, y_1, z_1) 代表轮廓曲线上的任意一点， (x_d, y_d, z_d) 代表导弹坐标。

同时，可得到该直线的一个方向向量 $\vec{m}$ 以及该方向向量所对应的模长 $\|\vec{m}\|$ ：

$$\begin{cases} \vec{m} = (x_1 - x_d, y_1 - y_d, z_1 - z_d) \\ \|\vec{m}\| = \sqrt{(x_1 - x_d)^2 + (y_1 - y_d)^2 + (z_1 - z_d)^2} \end{cases} \quad (10)$$

其次，根据导弹坐标以及直线向量可以给出该直线的参数形式以便后文分析建模：

$$\vec{L}(\rho) = \vec{d} + \vec{n}\rho \quad (11)$$

其中， $\vec{d} = (x_d, y_d, z_d)$ 为导弹实时坐标， $\vec{n}$ 为单位方向向量： $\vec{n} = \frac{\vec{m}}{\|\vec{m}\|} = (n_1, n_2, n_3)$ 。联立上式代入化简，即可得到 $\{x, y, z\}$ 各自独立的关系表达式如下。

$$\begin{cases} x = x_d + (x_1 - x_d)\rho \\ y = y_d + (y_1 - y_d)\rho \\ z = z_d + (z_1 - z_d)\rho \end{cases} \quad (12)$$

其中， ρ 代表直线的参数化变量，通过参数化直线可方便分析直线上任意点的位置。

此外，干扰弹爆炸后所产生的以 3m/s 匀速下降的烟幕云团，在模型中可视其为以 (x_b, y_b, z_b) 为原心，半径为 $R = 10m$ 的球体，故根据其实时坐标可给出云团的球面方程：

$$(x - x_b)^2 + (y - y_b)^2 + (z - z_b)^2 = R^2 \quad (13)$$

■ 代数判定推导

为判断直线是否经过球体，根据解析几何内容将直线参数方程代入云团球体公式：

$$(x_d + n_1\rho - x_b)^2 + (y_d + n_2\rho - y_b)^2 + (z_d + n_3\rho - z_b)^2 = R^2 \quad (14)$$

将几何关系准备部分所得的坐标关系代入上式展开化简，得到如下一元二次方程：

$$\begin{aligned} & (n_1^2 + n_2^2 + n_3^2) t^2 + 2[n_1(x_d - x_b) + n_2(y_d - y_b) + n_3(z_d - z_b)] t \\ & + [(x_d - x_b)^2 + (y_d - y_b)^2 + (z_d - z_b)^2 - R^2] = 0 \end{aligned} \quad (15)$$

同时，由于 $\vec{n} = (n_1, n_2, n_3)$ 为单位向量，单位向量一定存在关系 $n_1^2 + n_2^2 + n_3^2 = 1$ ，将该单位向量对应的代数关系代入上式化简，最终可得到以下化简后的一元二次方程：

$$at^2 + bt + c = 0 \quad (16)$$

其中，上式为一元二次方程的标准形式，其系数 a,b,c 各自所代表的数学含义如下。

$$\begin{cases} a = 1 \\ b = 2[n_1(x_d - x_b) + n_2(y_d - y_b) + n_3(z_d - z_b)] \\ c = [(x_d - x_b)^2 + (y_d - y_b)^2 + (z_d - z_b)^2 - R^2] \end{cases} \quad (17)$$

此方程有解表明直线与烟幕云团球体有交点，故本文选择采用一元二次的判别式来直观判断连线是否与烟幕云团球体有交点，其具体分类讨论后的判别情况如下所示。

$$\Delta = b^2 - 4ac$$

- 若 $\Delta > 0$ ，则表明有两个交点：连线 L 完全穿过了云团覆盖范围 A，为有效覆盖。
- 若 $\Delta = 0$ ，则表明有一个交点：连线 L 与云团覆盖范围 A 相切，为有效覆盖。
- 若 $\Delta < 0$ ，则表明没有交点：连线 L 与云团覆盖范围 A 没有交点，为无效覆盖。

其中，连线 L 与覆盖范围 A 的具体含义均已在判定核心逻辑规则图 3 中具体说明。

■ 目标函数

基于有效遮挡规则，本文对此建立一个示性函数 $I_{[\Delta]}$ 来为目标函数的构建做准备：

$$I_{[\Delta]} = \begin{cases} 1, & \text{若投影轮廓线上任意一点满足连线 } \Delta \geq 0 \\ 0, & \text{若投影轮廓线上任意一点满足连线 } \Delta < 0 \end{cases} \quad (18)$$

基于云团 20 秒消散机制，本文对此根据引爆时间 t_b 定义另一个示性函数 $I_{[t]}$ 如下：

$$I_{[t]} = \begin{cases} 1, & \text{若 } t - t_b \leq 20 \\ 0, & \text{若 } t - t_b > 20 \end{cases} \quad (19)$$

其中， $t - t_b$ 的约束表示当引爆后云团 A 超过有效时间则该函数为 0，反之则为 1。因此，将上述两个示性函数联立整合，以积分求和思想拟合出目标函数：

$$\int I_{[\Delta]} I_{[t]} dt \quad (20)$$

其中， $I_{[\Delta]}$ 用于判定某时刻是否遮蔽成功， $I_{[t]}$ 用于判定是否处于云团有效时间内。

5.1.4 模型总结

联立上述目标函数、示性函数及代数推导式，可得有效遮蔽时长计算模型如下：

$$\int I_{[\Delta]} I_{[t]} dt \quad (21)$$

$$s.t \left\{ \begin{array}{l} I_{[\Delta]} = \begin{cases} 1, & \text{若投影轮廓线上任意一点满足连线 } \Delta \geq 0 \\ 0, & \text{若投影轮廓线上任意一点满足连线 } \Delta < 0 \end{cases} \\ I_{[t]} = \begin{cases} 1, & \text{若 } t - t_b \leq 20 \\ 0, & \text{若 } t - t_b > 20 \end{cases} \\ at^2 + bt + c = 0 \quad \Delta = b^2 - 4ac \\ \begin{cases} a = 1 & b = 2[n_1(x_d - x_b) + n_2(y_d - y_b) + n_3(z_d - z_b)] \\ c = [(x_d - x_b)^2 + (y_d - y_b)^2 + (z_d - z_b)^2 - R^2] \end{cases} \\ (x - x_b)^2 + (y - y_b)^2 + (z - z_b)^2 = R^2 \quad \vec{n} = \frac{\vec{m}}{\|\vec{m}\|} = (n_1, n_2, n_3) \\ \vec{L}(\rho) = \vec{d} + \vec{n}\rho \quad \vec{d} = (x_d, y_d, z_d) \quad \vec{m} = (x_1 - x_d, y_1 - y_d, z_1 - z_d) \\ \begin{cases} x = x_d + (x_1 - x_d)\rho \\ y = y_d + (y_1 - y_d)\rho \\ z = z_d + (z_1 - z_d)\rho \end{cases} \begin{cases} z_d = z_{d0} - v_d t \sin \theta \\ x_d = x_{d0} - v_d t \cos \theta \\ y_d = 0 \\ x_d \in [0, x_{d0}] \end{cases} \\ \begin{cases} z_w = z_{w0} \\ x_w = x_{w0} - v_w t \\ y_w = 0 \\ x_w \in [0, x_{w0}] \end{cases} \begin{cases} z_g = z_{g0} - \frac{1}{2}g(t - t_{g0})^2 \\ x_g = x_{g0} - v_w(t - t_{g0}) \\ y_g = 0 \\ x_g \in [x_{g0}, x_{b0}] \end{cases} \begin{cases} z_b = z_{b0} - v_b(t - t_b) \\ x_b = x_{b0} \\ y_b = 0 \\ z_b \in [0, z_{b0}] \end{cases} \end{array} \right.$$

5.1.5 有效遮蔽时长计算

在本模型中，核心任务是判定连线 L 与云团覆盖范围 A 之间的相交关系，并确定有效遮蔽时长。为此本文设计了两类算法思路：解析法与采样法。首先，以解析法作为主要求解算法计算有效遮蔽时长。其次，以采样法求解模型得到结果作为结果验证和数值对照手段。最后，通过模拟不同采样思路求解模型，以展开结果的可靠性分析。

一、解析法求解	二、采样法验证
(1)方法：通过严格的数学公式和几何约束关系进行推导，直接判断在任意时刻连线L与云团覆盖范围A之间是否存在交点。 (2)核心判断依据：通过判别式$\Delta = b^2 - 4ac$判断是否存在交点： $\times \Delta < 0$：连线与球体无交点，该点位为无效遮挡。 $\checkmark \Delta \geq 0$：可进一步求解交点数，该点为有效遮挡。	(1)方法：通过对时间域进行离散化处理，逐步扫描时间点，判断在各个采样时刻连线与球体是否相交。 (2)核心判断依据：调用几何判定函数判断是否该点相交 PART1：时间离散化、固定步长dt、划分区域 PART2：针对采样点逐点判断
三、算法优劣势对比	
(1)解析法：计算结果的完整性高、计算结果精度高；不依赖于采样点的离散化与采样方式。 (2)采样法：易于算法实现、适用于结果验证；结果精度依赖步长，步长选取影响计算精度。	

图 6 解析-采样方法对比图

■ 解析法求解

在上述思路阐述的基础上，利用 Python 语言进行求解，具体的求解步骤如下：

Step1. 轨迹建模及几何关系推导

根据云团的动力学特性与上文模型的构建思路，量化球心 $C(t)$ 的运动模型，包括匀速平移阶段、抛物下落阶段及速度切换阶段，从而获得球体在任意时间 t 的空间位置。

将观察点 M 与目标点 Q 之间的连线参数化表示为： $X(s) = M + s(Q - M), s \in [0, 1]$ 。将该参数形式代入球体方程： $(X - C(t)) \cdot (X - C(t)) = R^2$ ，得到关于 s 的二次方程： $as^2 + bs + c = 0$ 。其中参数 $\{a = (Q - M)^2, b = 2(Q - M) \cdot (M - C), c = (M - C)^2 - R^2\}$ 。

Step2. 角度约束与全覆盖判定

在本题中对于圆柱体这种连续曲面的目标，需要判断云团覆盖范围是否能够完全覆盖其空间边界。故需通过建立球心到圆柱体各关键点的视锥角度约束，以确保球体视锥包含圆柱体顶点、底面圆周及中间关键截面，实现烟幕云团全覆盖真目标的预期效果。

Step3. 判定式分析与交点判定

通过关于 s 的二次方程的判别式 $\Delta = b^2 - 4ac$ 来判断是否存在有交点，分类如下：

- 若 $\Delta < 0$ ，说明连线与球体无交点，此情况下为无效遮挡。
- 若 $\Delta \geq 0$ ，可求解 $s_{1,2} = \frac{-b \pm \sqrt{\Delta}}{2a}$ ，以判断具体的交点数量，此情况为有效遮挡。

Step4. 动态时间扫描

将上述判定条件应用于时间域内的任意时刻，获得首次相交时间 t_{start} 与最终相交时间 t_{end} ，并计算遮挡持续时间： $T_{duration} = t_{end} - t_{start}$ 。同时，在这个遍历过程中采取**变时间步长遍历策略**，其基本思想是：根据观察点 $Q(t)$ 与球心 $C(t)$ 之间的瞬时距离 $d(t) = \|Q(t) - C(t)\|$ 动态调整时间步长 Δt [4]。当两者距离较近时，遮挡关系随时间变化更为敏感，因此选取较小的 Δt 以保证精度；当两者距离较远时，遮挡关系变化相对平缓，则采用较大的 Δt 以减少冗余计算，以上思想可形式化地表示为以下数学形式：

$$\Delta t(t) = \max(\Delta t_{\min}, \min(\Delta t_{\max}, k \cdot d(t))) \quad (22)$$

其中 k 为比例因子， $\Delta t_{\min}$ 与 $\Delta t_{\max}$ 分别为时间步长的下限和上限。

通过该策略能够在**保证求解精度**的同时显著**降低时间复杂度**，最终求解结果如下：

$$\left\{ \begin{array}{l} \text{有效遮挡时长: } 1.3808\text{s} \\ \text{第一次完全遮挡的时刻: } 8.0598\text{s} \\ \text{最后一次完全遮挡时刻: } 9.4406\text{ s} \end{array} \right.$$

■ 采样法验证

设总时间区间为 $[0, T]$ ，引入采样法作为辅助数值方法来验证解析法的正确性：

Step1. 时间离散化：按照固定步长 dt 将其划分为 $N = T/dt$ 个采样点。

Step2. 逐点判定

在每个采样时刻 t_i ，计算目标点的位置 $Q(t_i)$ 与球心位置 $C(t_i)$ 。针对每一时刻点调用模型中的几何判定函数来判断该情况下是否存在交点，以达到逐点判定的遍历效果。

Step3. 相交区间检测

当检测到从“不相交”状态切换为“相交”状态时，记录该时刻为 t_{start} ；当检测到从“相交”切换为“不相交”时，记录该时刻为 t_{end} 。最终即可得到相交区间及有效遮蔽时长，有效遮蔽时长的结果为 **1.3808s**，与解析法结果一致，初步验证出结果的准确性。

■ 结果鲁棒性分析

为了验证上文计算结果的可靠性，本文选择改变采样点数量并施加合理扰动，探究其对遮蔽时间的影响，结果如右图所示：

▷ 随着采样点数量的增加，或在采样点位置上引入不同幅度的随机扰动，最终遮蔽时间始终保持在几乎相同的水平面。

▷ 所提出的基于采样法的遮蔽时间计算方法在数值稳定性方面表现良好，能够有效避免因采样点分布密度不足或扰动噪声引入而造成的显著偏差。

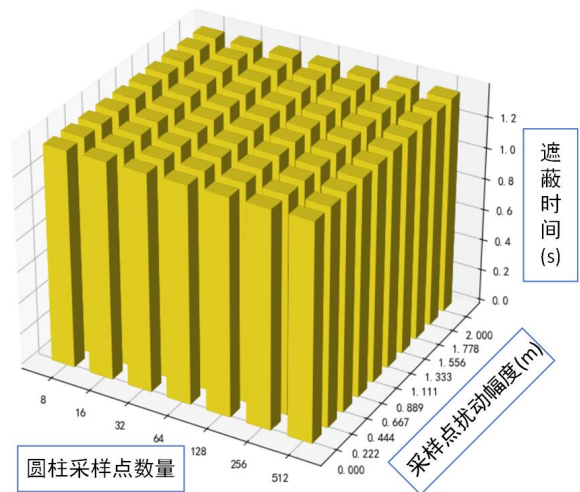


图 7 采样法结果可靠性分析

5.2 问题二：单机单次投弹策略优化模型

在问题一的基础上，取消无人机飞行轨迹与投弹策略的具体限制，需以最大化有效遮蔽时长为目标求解无人机具体飞行策略及干扰弹起爆策略(图 8 所示)。故本文选择在上文遮蔽时长计算模型的基础上，适应性调整运动轨迹方程并施加相关约束条件来构建模型，最后以创新性融合优化算法而非直接搜索算法来求解模型并展开灵敏度分析。

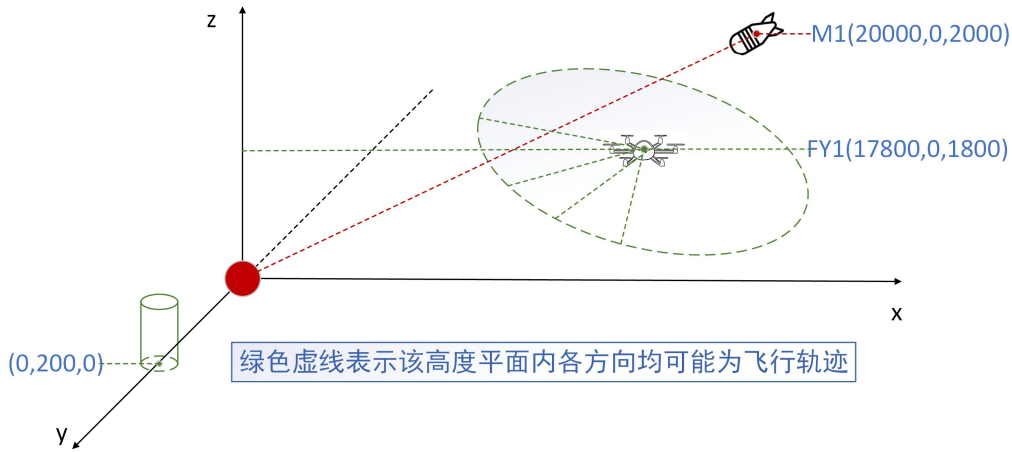


图 8 问题二坐标系下求解背景示意图

5.2.1 投弹策略优化模型构建

■ 模型准备

本题无人机的航线设定更加自由，需求解出无人机的飞行方向、飞行速度 v_w 、干扰弹投放点 (x_{g0}, y_{g0}, z_{g0}) 与起爆点 (x_{b0}, y_{b0}, z_{b0}) 。在以上四个角度中飞行方向最难量化表示，故本文选择先定义一个关键参数：飞行角度 α 来辅助量化飞行方向。

• **飞行角度 α** : 在高度 z_w 等高固定平面内，在第 i 时刻时，速度方向 v_i 与 x 轴正向逆时针的夹角，取值范围约束 $\alpha \in [0, 360^\circ]$ ，该参数具体的几何关系演示详见右图所示。

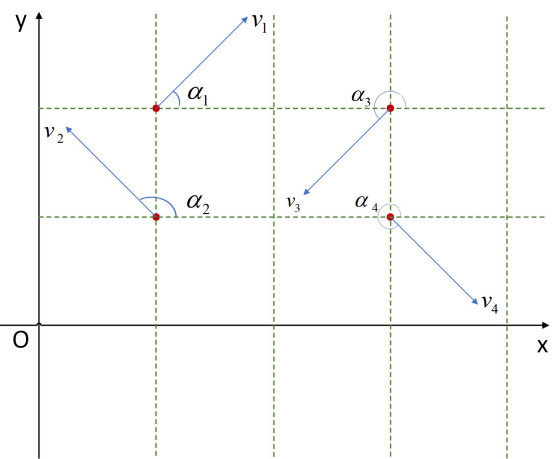


图 9 飞行角度 α 几何演示

■ 运动轨迹及约束适应性调整

无人机的轨迹方程与坐标变化方程相对于问题一有所变化。此时，选定固定合适的飞行角度 α 与固定合适的速度 v_w 是延长有效遮蔽时长的关键，故先初步定性分析如下：

- 若飞行角度致使无人机飞行过程太偏离真目标点，则会使投弹的遮蔽时间大大减弱。

- 若飞行角度致使无人机飞行过程太靠近真目标点，则有可能使得遮蔽能力大大削弱。
- 若飞行速度设置不合适会导致烟幕干扰弹被投放时初始速度不利于后续遮蔽目标。

基于上文问题一模型基础，适应性调整后可得飞行轨迹关系及坐标变化方程如下：

$$\text{飞行轨迹} = \begin{cases} y = \tan \alpha (x - 17800) \\ z = 1800 \\ \alpha: \text{飞行角度} \\ \alpha \in [0, 360^\circ] \end{cases} \iff \text{坐标变化方程} \begin{cases} y = v_w t \sin \alpha \\ x = v_w t \cos \alpha + 17800 \\ z = 1800 \\ \alpha \in [0, 360^\circ] \\ v_w \in [70\text{m/s}, 140\text{m/s}] \end{cases} \quad (23)$$

其中，飞行角度 α 与速度 v_w 的约束范围为题干所给定，不可在后序模型中有改变。

此外，干扰弹的投放点、引爆点及云团运动等研究均从问题一中的 xz 平面转移到了与 x 轴正向夹角为 α 的平面 $y = \tan \alpha (x - 17800)$ 上。因此，在模型一的基础上适应性调整后可知，对于干扰弹投放点 (x_{g0}, y_{g0}, z_{g0}) ，除式 (3) 关系外还需满足以下条件：

$$\begin{cases} y_{g0} = \tan \alpha (x_{g0} - 17800) \\ z_{g0} = 1800 \end{cases} \quad (24)$$

同理，其余运动的核心规律不变，在上文基础上适应性调整即可。此外，本题目标是尽量使遮蔽时间更长，因此本问的**目标函数是最大化遮蔽时间函数**： $\max \int I_{[\Delta]} I_{[t]} dt$ 。

联立目标函数与所有调整过后的代数关系式，可得到**单机单次投弹策略优化模型**：

$$\max \int I_{[\Delta]} I_{[t]} dt \quad (25)$$

$$s.t \begin{cases} I_{[\Delta]} = \begin{cases} 1, \text{任意点满足连线} \Delta \geq 0 \\ 0, \text{任意点满足连线} \Delta < 0 \end{cases} & I_{[t]} = \begin{cases} 1, t - t_b \leq 20 \\ 0, t - t_0 > 20 \end{cases} \\ \begin{cases} at^2 + bt + c = 0 \\ \Delta = b^2 - 4ac \end{cases} & \begin{cases} a = 1 & b = 2[n_1(x_d - x_b) + n_2(y_d - y_b) + n_3(z_d - z_b)] \\ c = [(x_d - x_b)^2 + (y_d - y_b)^2 + (z_d - z_b)^2 - R^2] \end{cases} \\ (x - x_b)^2 + (y - y_b)^2 + (z - z_b)^2 = R^2 & \vec{n} = \frac{\vec{m}}{\|\vec{m}\|} = (n_1, n_2, n_3) \\ \vec{L}(\rho) = \vec{d} + \vec{n}\rho & \vec{d} = (x_d, y_d, z_d) \quad \vec{m} = (x_1 - x_d, y_1 - y_d, z_1 - z_d) \\ \begin{cases} z_d = z_{d0} - v_d t \sin \theta \\ x_d = x_{d0} - v_d t \cos \theta \\ y_d = 0 \quad x_d \in [0, x_{d0}] \end{cases} & \begin{cases} z_w = z_{w0} = 1800 \\ x_w = x_{w0} + v_w t \cos \alpha \\ y_w = v_w t \sin \alpha \end{cases} & \begin{cases} z_g = z_{g0} - \frac{1}{2}g(t - t_{g0})^2 \\ x_g = x_{g0} + v_w(t - t_{g0}) \cos \alpha \\ y_g = v_w t \sin \alpha \end{cases} \\ x_b = x_{b0} \quad y_b = y_{b0} \quad z_b = z_{b0} - vb(t - t_b) & \alpha \in [0, 360^\circ] \quad v_w \in [70\text{m/s}, 140\text{m/s}] \end{cases}$$

5.2.2 创新性融合优化算法求解

由于目标函数复杂、非线性且难以获得解析梯度，单一局部优化容易陷入局部最优，而单一全局优化收敛缓慢且精度不足。故本文选择不使用直接遍历搜索的硬算解法，而是采用 **遗传算法 (GA) 与 Nelder-Mead(NM) 单纯形法相结合的混合优化策略**，实现全局搜索与局部精细收敛的互补，该创新性融合优化算法的整体流程思路如下所示 [5]。

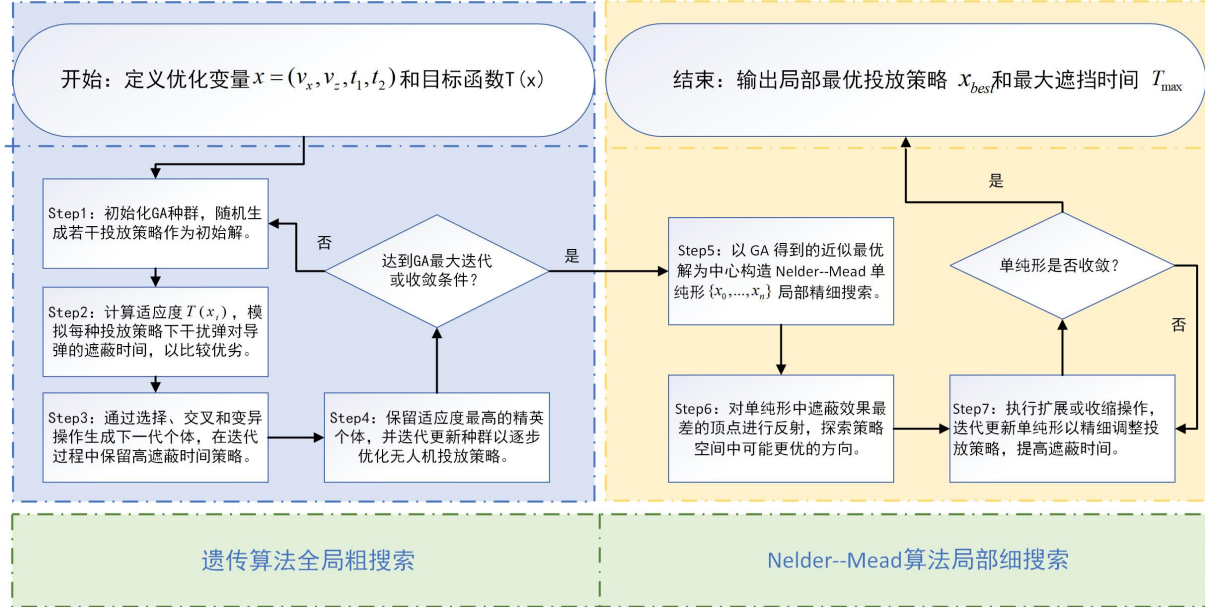


图 10 创新性融合算法流程图

首先，遗传算法在全局范围内搜索各类解空间，获取适应度较大的近似最优策略区域，避免算法陷入局部极值。随后结合 Nelder-Mead 单纯形法，在局部区域进行精细搜索。该算法流程体现出“全局粗调 + 局部精细优化”的思想，具体的实现步骤如下：

Step1. GA 种群初始化、适应度计算

随机生成种群个体，每个个体表示一组投放策略参数 $\mathbf{x}_i$ ，在其各自边界 $[lb_j, ub_j]$ 内初始化。同时，对每组策略参数 $\mathbf{x}_i$ 展开模拟，计算遮蔽时间，并以此作为适应度值：

$$\text{fitness}_i = T(\mathbf{x}_i) \quad (26)$$

其中， fitness_i 的适应度计算，用于为后序选择、交叉、变异等操作提供遗传父代。

Step2. 优化筛选 (选择、交叉、变异)

首先展开选择操作，采用轮盘赌的选择方法从当前种群中选择适应度较高的个体作为父代。适应度越高的投放策略越可能被选中，以保证高遮蔽时间策略有更多机会进入下一代。令符号 O 来表示在 Step1 中随机生成的种群个体，则每个个体被选中的概率为：

$$P_i = \frac{\text{fitness}_i}{\sum_{j=1}^O \text{fitness}_j} \quad (27)$$

其次展开交叉操作，随机配对父代 $\mathbf{x}_p$ 和 $\mathbf{x}_q$ ，生成子代 $\mathbf{x}_c$ 如下式所示。通过以下算术组合操作可实现父代策略特征的混合，既能保留优良基因，又可增加策略可能性。

$$\mathbf{x}_c = w\mathbf{x}_p + (1 - w)\mathbf{x}_q, \quad w \sim U(0, 1) \quad (28)$$

最后展开变异操作，将每个子代以概率 p 对参数进行变异，以避免筛选过早收敛：

$$x'_{c,j} = x_{c,j} + \delta_j, \quad \delta_j \sim U(-\sigma_j, \sigma_j) \quad (29)$$

其中， $x_{c,j}$ 为子代第 j 个参数， σ_j 为该参数的变异幅度，变异后若超出边界则截断。

Step4. 精英保留与迭代

在重复组合迭代过程中，采用精英保留的策略，即选择适应度值最高的 $E\%$ 个体进行保留，且最终当达到预设的最大迭代次数或连续若干代遮蔽时间增量低于收敛阈值时，遗传 GA 算法终止迭代，输出结果进入 Nelder-Mead 算法部分展开进一步优化。

Step5. 单纯形构造与顶点排序

在遗传算法阶段得到的全局近似最优解 $\mathbf{x}_{GA}^*$ 基础上，以其为中心构建初始单纯形 $\{\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_n\}$ ，其中每个顶点 $\mathbf{x}_k$ 对应一组飞行策略。其次，对单纯形各顶点展开模拟，计算每个顶点对应投放策略的有效遮蔽时间 $T(\mathbf{x}_k)$ ，并根据时间长短排序： $T(\mathbf{x}_0) \geq T(\mathbf{x}_1) \geq \dots \geq T(\mathbf{x}_n)$ ，得到单纯形中的最佳顶点 $T(\mathbf{x}_0)$ 。

Step6. 局部调整 (反射、扩展、收缩)

在单纯形构造并排序后，对遮蔽效果最差的顶点 $\mathbf{x}_w$ 进行反射，以探索在策略空间中遮蔽效果更佳的方向。反射操作的计算公式为： $\mathbf{x}_r = \mathbf{x}_c + \alpha(\mathbf{x}_c - \mathbf{x}_w)$ ， α 为反射系数。

若反射点 $\mathbf{x}_r$ 对应的遮蔽时间 $T(\mathbf{x}_r)$ 优于当前最优顶点 $T(\mathbf{x}_b)$ ，则进行扩展操作，进一步探索更高效的投放策略： $\mathbf{x}_e = \mathbf{x}_c + \gamma(\mathbf{x}_r - \mathbf{x}_c)$ 。其中， γ 为扩展系数且满足 $\gamma > 1$ 。

若反射未能改善遮蔽效果，即 $T(\mathbf{x}_r) < T(\mathbf{x}_b)$ ，则对单纯形进行收缩操作以逐步集中搜索最优投放参数： $\mathbf{x}_k = \mathbf{x}_b + \beta(\mathbf{x}_i - \mathbf{x}_b)$ 。其中， β 为收缩系数且需满足 $0 < \beta < 1$ 。

Step7. 收敛判定与局部最优策略确定

当单纯形的尺寸 $\|\mathbf{x}_{\max} - \mathbf{x}_{\min}\|$ 足够小 (即 $\|\mathbf{x}_{\max} - \mathbf{x}_{\min}\| < \delta$)，或连续若干迭代中的改进 ΔT 低于预设阈值 ϵ 时 (即 $\Delta T < \epsilon$)，此时可认为该融合算法流程已经收敛。在此时停止迭代，并将当前单纯形中有效遮蔽时间最长的顶点 $\mathbf{x}_{\text{best}}$ 输出作为最优投放策略 [3]。

最终在执行上述 GA-NM 融合算法流程后，将问题二的求解结果整合如下所示：

$$\left\{ \begin{array}{l} \text{飞行角度 } \alpha: 178.28^\circ \\ \text{飞行速度 } v_w: 77.8098 \text{ m/s} \\ \text{投放点坐标: } (17750.99, 1.47, 1800.00) \\ \text{起爆点坐标: } (17578.40, 6.65, 1775.87) \end{array} \right. \Rightarrow \{\text{有效遮挡时长: } 4.5128 \text{ s}\}$$

5.2.3 灵敏度分析

本文选择以灵敏度分析的形式展开模型的可靠性分析，通过对最优解附近的关键参数展开单因素扰动分析，系统地考察无人机速度分量 (v_x 、 v_y)、干扰弹投放时间以及起爆时间四者对遮蔽时间的影响。该方法能够直观地揭示各个参数在局部范围内的灵敏度差异，从而判断不同决策变量对整体模型的干扰效果，最终得到以下拟合图像结果。

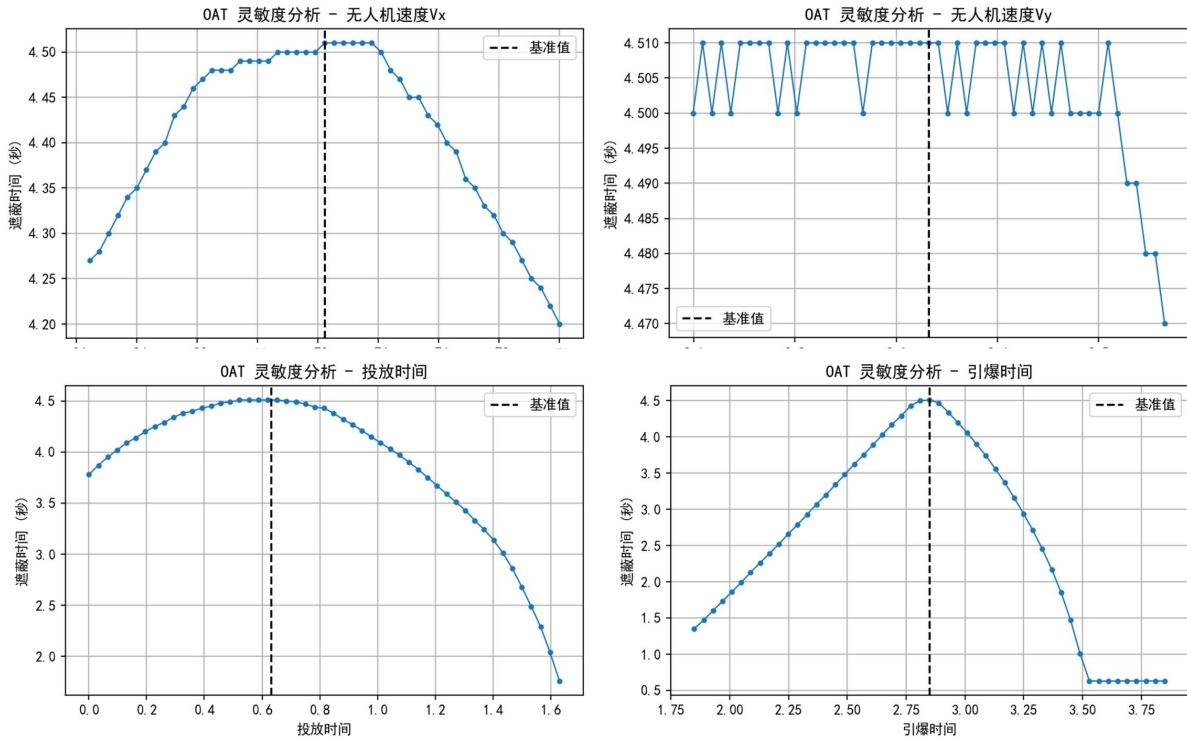


图 11 灵敏度分析结果

- **无人机速度 v_x :** 当 v_x 在基准值上下浮动时，遮蔽时间呈现为先升高后下降。遮蔽时间在 4.2-4.5 s 范围内波动但不够连续，表明 v_x 对结果有一定敏感性，但影响有限。
- **无人机速度 v_y :** 波动极微小表明 v_y 对遮蔽时间的影响可以忽略，灵敏度极低。
- **投放时间:** 随着投放时间从 0 s 增大到 1.6 s，遮蔽时间呈现明显的“先上升后下降”趋势且曲线连续。同时在 0.6s 左右时，遮蔽时间达到峰值附近，过早或过晚投放均会导致遮蔽时间缩短，表明投放时间的灵敏度极高，存在最佳投放区间。
- **引爆时间:** 随着引爆时间延后，遮蔽时间增加并在 2.8 s 左右达到峰值随后迅速下降，表明引爆时机对遮蔽效果至关重要，若引爆过早或过晚，遮蔽时间会显著降低。

综上所述，速度参数主要影响烟幕云团的空间分布位置，而时间参数则直接决定了烟幕形成与导弹飞行路径的匹配程度，二者在作用机理和敏感性上存在显著差异。且**投放时间**和**引爆时间**对最终有效遮蔽时长的影响最为显著，具有明显的最佳区间和非线性效应。因此，在后续问题研究中可优先关注投放与引爆的时间控制，其灵敏度极高。

5.3 问题三：单机多次投弹策略优化模型

在问题三中，相较于问题二增加了无人机可投的烟幕干扰弹数量，这必然有利于对真目标的保护，即可延长有效遮挡时长。本文在此选择遵从上文所提出的核心思想来构建模型：根据射影曲线与导弹的连线是否穿过有效遮蔽区域来作为最终的判断准则。

5.3.1 多投弹几何和时间约束分析

首先,由于无人机以方向偏角 α 在固定高度上直线匀速飞行,因此投弹位置 (x_{1i}, y_{1i}, z_{1i}) 与引爆位置 $(x'_{1i}, y'_{1i}, z'_{1i})$ 都必须在所属无人机 FY1 的航线平面内且发生在地面及以上:

$$\begin{cases} (x_{1i}, y_{1i}, z_{1i}) \in y = \tan \alpha (x - 17800) \\ (x'_{1i}, y'_{1i}, z'_{1i}) \in y = \tan \alpha (x - 17800) \\ z_{1i} > 0, z'_{1i} > 0, (i = 1, 2, 3) \end{cases} \quad (30)$$

其次，根据同一架无人机投弹时间间隔不得超过 1s 的限制，三个干扰弹的投弹位置会有以下几何约束关系：先被投放的干扰弹的投放点与无人机初始位置的距离更近：

$$\|(x_{1i}, y_{1i}, z_{1i}) - (x_w, y_w, z_w)\| < \|(x_{1j}, y_{1j}, z_{1j}) - (x_w, y_w, z_w)\|, (i = 1, 2, 3; i < j) \quad (31)$$

其中，参数右下侧角标中的数字大小顺序表示投放顺序先后，即 1 表示最先投放。

此外，该投弹约束可转换为相邻两个投放点距离不小于无人机行驶 1s 所行的距离：

$$\|(x_{1i}, y_{1i}, z_{1i}) - (x_{1(i+1)}, y_{1(i+1)}, z_{1(i+1)})\| \geq v_w \cdot 1, (i = 1, 2, 3) \quad (32)$$

5.3.2 多烟幕云团组合情况分析

本题在加入多投弹情况后，云团遮蔽区域会明显增大，但当无人机投放多枚烟幕干扰弹时，可能会存在因引爆时间不一导致的烟幕云团部分区域部分重叠的情况，此时遮蔽区域就不能再简单的抽象为一个球体，并且后续判断遮蔽是否成功也不能单一根据射影曲线上点与导弹连线是否能全部穿过某个云团而决定，因为可能存在两个烟幕云团共同分担遮蔽任务的情况，如右图所示。

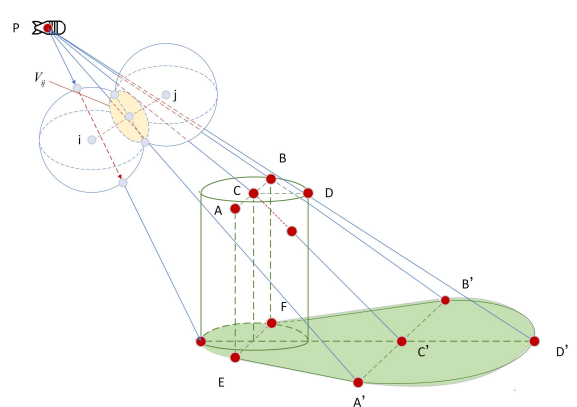


图 12 烟幕云团重叠示意图

当两个烟幕云团产生部分重叠时,二者由于烟幕云团匀速下降的特性会成一个整体**相对静止**匀速下降。同时,分析重叠情况可知,云团 i 与云团 j 发生重叠的开始时间 t_N 必然是其中更晚引爆的烟幕干扰弹的引爆时间,即可用以下数学形式来表示 t_N :

$$t_N = \max \{t'_{1i}, t'_{1i}\} \quad (33)$$

同时,相较于云团不重叠的情况不同,当云团重叠时不可直接拆分为问题一的情况。云团重叠部分 V_{ij} 的遮蔽时间与更晚引爆的云团一致(为最初 20s)。因此,当两个烟幕云团发生重叠时,整体可以拆分为一个减去重叠部分的残缺球体与一个完整球体。

在上文理论分析的基础上,可定义有关判定烟幕云团组合的示性函数 $I_{ij}(t)$ 如下:

$$I_{ij}(t) = \begin{cases} 1, t \text{时刻烟幕云团} i \text{与} j \text{发生组合} \\ 0, t \text{时刻烟幕云团} i \text{与} j \text{未发生组合} \end{cases} \quad (34)$$

- 对于未发生组合情况的云团,其遮蔽区域 $A_{1i}(t)$ 是 t 时刻以其为圆心的球体。

$$A_{1i}(t) \begin{cases} \text{区域: } A_{1i}(x_b, y_b, z_b) \text{ 球} \\ \text{有效时间: } t \in [t_{1i}, t_{1i} + 20] \end{cases} \quad (35)$$

- 对于发生组合的云团 i 与 j , 其与未组合的差异主要在于残缺球体 A'_{1i} 的处理。

$$A'_{1i}(t) \begin{cases} \text{区域: } A_{1i}(x_b, y_b, z_b) \text{ 球} - V_{ij} \\ \text{有效时间: } t \in [t_N, 20 - (t_N - t_{1i})] \end{cases} \quad (36)$$

综上所述,基于上述遮蔽区域分析可知,烟幕云团未组合时遮蔽区域为 $A_{1i}(t)$; 烟幕云团发生组合时的整体遮蔽区域为 $A_{1j}(t) + A'_{1i}(t)$ 。故根据上文所定义的示性函数 $I_{ij}(t)$, 可得到关于描述存在区域组合情况下的烟幕云团 i **遮蔽区域通用表示式**如下。

$$\Omega_i(t) = [A_{1j}(t) + A'_{1i}(t)] I_{ij}(t) + A_{1i}(t) [1 - I_{ij}(t)], (i, j = 1, 2, 3, i \neq j) \quad (37)$$

5.3.3 遮蔽判断 Δ 准则更新: Ω 准则

根据上文的遮蔽区域变化分析与本文遵循的核心判断准则,在此更新上文的遮蔽判断 Δ 准则为 Ω 准则。在任务开始后任意 t 时刻, Ω 准则的具体定义形式整理如下所示。

- $\Omega \geq 0$: 射影曲线上的点与导弹的连线全部穿过 $\Omega_i(t)$ 组成的遮蔽区域,为有效遮挡。
- $\Omega < 0$: 存在连线未穿过 $\Omega_i(t)$ ($i = 1, 2, 3$) 组成的遮蔽区域,为无效遮挡。

5.3.4 反演推导求解目标策略分析

本文选择在此对“由起爆点反向推导投弹点、无人机飞行速度和方向”的策略展开**反演机理分析**,以构建出完整的多策略选择优化模型体系,为后文更复杂的投弹情景求

解作准备。

由于无人机的行驶方向与速度在初始设定后不变，且烟雾干扰弹的各种运动和操作在单一平面内完成，故分析可知当烟幕干扰弹的最优引爆点 (x_b, y_b, z_b) 被确定时，无人机的飞行方向也将确定，其飞行角度 α 存在以下数学关系式：

$$\alpha = \begin{cases} \arccos\left(\frac{x_b-17800}{y_b}\right), & y_b \geq 0 \\ 2\pi - \arccos\left(\frac{x_b-17800}{y_b}\right), & y_b < 0 \end{cases} \quad (38)$$

但仅通过平抛运动和无人机匀速运动规律反推无人机速度是不够的。若投放至引爆点的时间无约束，会存在大量可到达轨迹；而过程时间的不确定性，会导致烟幕干扰弹即便到达最优引爆点，也无法处于最佳遮挡时段。因此，本文后续将围绕**引爆时间**的确定展开，以准确反推投放位置与无人机速度，具体如何确定起爆时间 t_b 详见下文分析：

对于某一起爆点 (x_b, y_b, z_b) 产生的烟幕云团，该烟雾云团在引爆时间 t_b 开始，竖直向下匀速运动，且其下落时所经过的区域范围所对应的烟雾有效时间 T 最长为 20s：

$$T = \min\left\{\frac{z_b}{v_b}, 20\right\} \quad (39)$$

▷ 烟幕弹开始下落至 T 时，这个过程一定对应存在导弹轨迹上一段以 t_b 时刻开始经过 T 时间飞行的线段，若能找到该线段位置，就可根据端点位置以及导弹的速度 v_d 求解出引爆时刻 t_b ，再根据最优引爆点位置以及运动情况即可反推投放点位置以及飞机的速度。假设 S_1 为对应时段的导弹运动路径，那就可以获得这段过程中的具体遮蔽情况，令 P_1 是导弹飞行过程中真目标

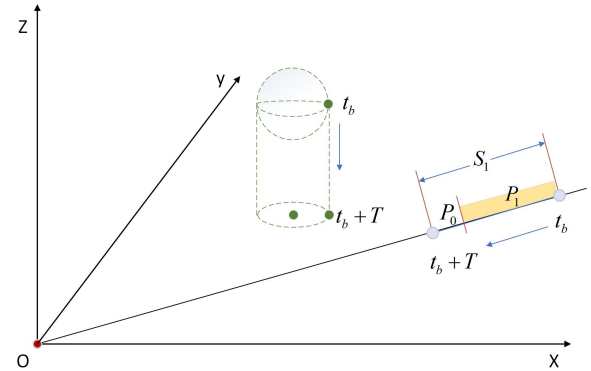


图 13 起爆时间确定论证示意图

被成功遮蔽的过程，那么该烟幕云团遮蔽时间计算可得为 P_1/v_w 。因此，遮蔽时长越长表示对应的 P_1 长度越长，引爆点位置决定 T 从而决定要寻找的对应时段的 S_1 的长度，而 P_1 长度则反映遮蔽情况，据此两点可以决定引爆点位置以及对应时段的导弹路径从而去实现最优遮蔽。之后根据前文反演推理理论就可以找寻到最优引爆点以及相应的引爆时间以及无人机飞行方向和速度。

因此，可得到目标函数的一组等效关系如下，二者分别代表两种不同的求解策略：

$$\text{正向搜索策略 } \max \int_{I[\Delta]} I[t] dt \iff \text{反向推导策略 } \max P_1$$

其中， P_1 为导弹飞行过程中真目标被成功遮蔽时段所对应的导弹路径长。

5.3.5 模型总结

联立目标函数与所有调整过后的代数关系式，可得到**单机多次投弹策略优化模型**：

$$\begin{aligned} & \max \int I_{[\Delta]} I_{[t]} dt \quad (40) \\ s.t. & \begin{cases} I_{[\Delta]} = \begin{cases} 1, \text{任意点满足连线} \Delta \geq 0 \\ 0, \text{任意点满足连线} \Delta < 0 \end{cases} & I_{[t]} = \begin{cases} 1, t - t_b \leq 20 \\ 0, t - t_0 > 20 \end{cases} \\ \begin{cases} at^2 + bt + c = 0 \\ \Delta = b^2 - 4ac \end{cases} & \begin{cases} a = 1 & b = 2[n_1(x_d - x_b) + n_2(y_d - y_b) + n_3(z_d - z_b)] \\ c = [(x_d - x_b)^2 + (y_d - y_b)^2 + (z_d - z_b)^2 - R^2] \end{cases} \\ (x - x_b)^2 + (y - y_b)^2 + (z - z_b)^2 = R^2 & \vec{n} = \frac{\vec{m}}{\|\vec{m}\|} = (n_1, n_2, n_3) \\ \vec{L}(\rho) = \vec{d} + \vec{n}\rho & \vec{d} = (x_d, y_d, z_d) \quad \vec{m} = (x_1 - x_d, y_1 - y_d, z_1 - z_d) \\ \begin{cases} z_d = z_{d0} - v_d t \sin \theta \\ x_d = x_{d0} - v_d t \cos \theta \\ y_d = 0 \quad x_d \in [0, x_{d0}] \end{cases} & \begin{cases} z_w = z_{w0} = 1800 \\ x_w = x_{w0} + v_w t \cos \alpha \\ y_w = v_w t \sin \alpha \end{cases} & \begin{cases} x_{b,k} = x_{b0,k} \\ y_{b,k} = y_{b0,k} \\ z_{b,k} = z_{b0,k} - v_{b,k}(t - t_{b,k}) \end{cases} \\ \begin{cases} z_{g,k} = z_{g0,k} - \frac{1}{2}g(t - t_{g0,k})^2 \\ x_{g,k} = x_{g0,k} + v_w(t - t_{g0,k}) \cos \alpha \\ y_{g,k} = v_w t \sin \alpha \end{cases} & \begin{cases} (x_{1i}, y_{1i}, z_{1i}) \in y = \tan \alpha (x - 17800) \\ (x'_{1i}, y'_{1i}, z'_{1i}) \in y = \tan \alpha (x - 17800) \\ z_{1i} > 0, z'_{1i} > 0, (i = 1, 2, 3) \end{cases} \\ \|(x_{1i}, y_{1i}, z_{1i}) - (x_w, y_w, z_w)\| < \|(x_{1j}, y_{1j}, z_{1j}) - (x_w, y_w, z_w)\|, & (i = 1, 2, 3; i < j) \\ \alpha \in [0, 360^\circ] \quad v_w \in [70m/s, 140m/s] \quad k \in \{1, 2, 3\} \\ \|(x_{1i}, y_{1i}, z_{1i}) - (x_{1(i+1)}, y_{1(i+1)}, z_{1(i+1)})\| \geq v_w \cdot 1, & (i = 1, 2, 3) \end{cases} \end{aligned}$$

5.3.6 反演计算-正向搜索验证

经上文反演推导求解目标策略的分析可知，反演计算策略相较于正向搜索策略，具有明显的计算效率优势与逻辑适用广泛性。当问题从单机单次投弹逐步深化至单机多次投弹，可以逐步感受到正向搜索算法策略的局限性，其在求解过程中极易陷入局部最优解。因此本文选择以反演策略作为主要计算思路，将正向搜索策略作为辅助验证。

Step1. 反演策略准备

- **空间网格化**：在题目可能涉及的最大空间范围中 ($x=[0,20000], y=[-3000,3000], z=[0,2100]$) 选定步长 $\{dx=0.1, dy=0.1, dz=0.1\}$ 来将坐标系空间网格化处理。
- **导弹轨迹反演**：在给定导弹轨迹 $M(t)$ 的模拟情况下，将时间离散化处理并在这过程中取时间步长 $dt=0.01$ 。从末端时刻 $t_{\max}$ 向前遍历，依次选取候选引爆时刻 t_b 。

Step2. 最优引爆点反演搜索

- 对于每一个 t_b ，计算确定导弹对应位置 $X_b = M(t_b)$ 。
- 计算导弹位置与圆柱体构成的视锥体中所有的空间坐标。
- 通过 GA-NM 算法搜索视锥体中不共面的四个点，进而确定球体的初步中心坐标。
- 在搜索后的坐标空间中展开时间模拟，其中，适应度函数为单个球体遮盖时间。
- 搜索后的结果为当前时间 t_b 下最优的球体中心坐标。

根据以上具体搜索策略对每个时间点展开计算模拟，可得到每个时间点所对应的最优球体中心坐标。在此基础上可直接反向递归出烟雾弹投掷坐标及其时间以及起爆时间，进而可推导计算出无人机的飞行速度，最终实现反演搜索策略的预期求解效果。

Step3. 正向搜索策略验证

在上文单机单次投弹情形求解思路的基础上展开适应性调整，将变量优化拓展为：

$$\mathbf{x} = (v, t_{p1}, t_{p2}, t_{p3}, t_{b1}, t_{b2}, t_{b3}),$$

其中 v 为无人机飞行速度， t_{pi} 为第 i 枚烟幕干扰弹投放时刻， t_{bi} 为对应起爆时刻。

同时，将目标函数适应性调整为三枚烟幕干扰弹的总有效遮蔽时长 (即通过时间模拟导弹飞行及三枚烟幕干扰弹的运动轨迹，计算遮蔽区间并集长度作为优化目标)：

$$\max_{\mathbf{x}} T(\mathbf{x}) = \left| \bigcup_{i=1}^3 [t_{i,\text{start}}, t_{i,\text{end}}] \right| \quad (41)$$

除此之外，根据问题二相同的算法思路，经过 GA-NM 融合算法计算求解，得到正向搜索策略下对应的投弹策略与总有效遮蔽时长，用以对比验证反演策略的优劣性。

Step4. 计算结果对比分析

将反演策略与正向搜索策略各自计算的投弹策略结果及运行时间整合如下所示：

表 1 不同策略计算结果对比

策略选择	总有效遮蔽时长 (s)	投弹 1 时刻 (s)	投弹 2 时刻 (s)	投弹 3 时刻 (s)	代码总运行时间 (s)
反演推导	6.5517	0.0824	2.4906	4.8836	93.58
正向搜索	6.3412	0.2912	2.3742	4.5514	178.77

通过以上策略计算结果对比可知，反演策略的计算结果与计算效率均优于正向搜索策略，故最终选定反演策略对应的投弹策略为最优方案，导出该方案的详细信息如下。

$$\begin{cases} \text{飞行角度 } \alpha: 179.51^\circ \\ \text{飞行速度 } v_w: 114.2299 \text{ m/s} \end{cases} \Rightarrow \{\text{有效遮挡时长: } 6.5517 \text{ s}\}$$

表 2 反演策略具体投弹方案

干扰弹编号	各自有效遮蔽时长结果 (s)	投放坐标:x	y	z	起爆坐标:x	y	z
1	3.4322	17791.20	0.08	1800.00	17417.40	3.30	1747.52
2	3.2634	17515.19	2.46	1800.00	17307.60	4.25	1783.82
3	1.2704	17242.87	4.81	1800.00	17211.33	5.08	1799.63

5.3.7 仿真验证

针对该单机多次投弹的复杂问题，本问在优化得到的投放策略基础上，设计了仿真模型，来验证协同干扰效果，并对无人机的飞行轨迹与导弹遮蔽过程进行可视化分析。

• **仿真方法：**通过设计多环节仿真模型实现目标：无人机运动按优化结果计算水平平面位置随时间的变化，烟幕干扰弹投放后沿初速度方向运动且起爆后云团半径随时间增长，导弹按匀速直线运动模拟位置更新，每个时间步通过判断导弹与烟幕弹的距离是否小于云团半径来确定是否计入遮蔽时间。

• **仿真可视化与结果分析：**仿真检验后得到的结果整理如下所示，将该结果与上文反演策略计算结果对比可知，本文选取策略的计算效果**优于**仿真结果，符合模拟预期。

干扰弹 1 干扰时长：3.43s；遮挡时间区间：4.66-8.09s

干扰弹 2 干扰时长：3.09s；遮挡时间区间：6.86-9.95s

干扰弹 3 干扰时长：1.09s；遮挡时间区间：9.93-11.02s

⇒ 总有效遮挡时长：6.36s

同时，拟合生成仿真检验的可视化结果，图表中的三片区域分别表示了三个烟雾弹对目标进行了有效遮蔽，对应的横坐标长度表示为烟雾弹的有效遮蔽时长，且虚线部分表示了两个干扰弹的遮蔽效果**存在重叠情况**，为后文讨论遮挡情况分类作好铺垫。

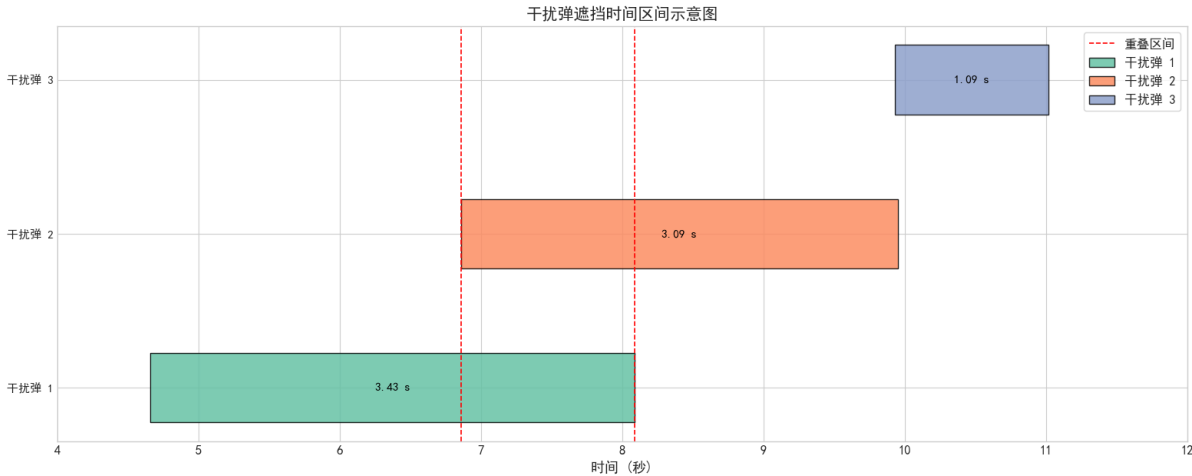


图 14 仿真检验可视化结果

• **仿真目的：**验证三方面内容：一是多枚烟幕干扰弹在不同投放时间与起爆时间下的遮蔽效果，二是导弹飞行过程中的被遮蔽时间段，三是无人机飞行轨迹及烟幕弹投放位置对遮蔽效果的影响。

综上所述，此次仿真不仅验证了求解结果的可靠性。同时从遮盖时间区间可以看出干扰弹的遮盖效果存在重叠，这会使得干扰弹的利用率有所降低，为后两问的求解提供了优化方向，在整体效果不变的前提下，**尽可能降低两个烟雾弹遮蔽的重叠时间，达到单个干扰弹有效遮蔽利用的最大化。**

5.4 问题四：多机单次投弹策略优化模型

本题与问题三均需投放三枚干扰弹，相较于问题三对三枚干扰弹的空间约束与时间约束，问题四的三枚干扰弹在空间与时间上的可行范围更灵活。此外，本题的场景可视作为问题二的泛化情况。具体而言，模型设定中三枚烟幕干扰弹需在三架无人机各自的飞行平面内完成投放与引爆，且此场景下不存在投放时间间隔限制。故本文将依次展开约束条件更新、云团相对位置分析、决策机理分析，利用**决策优劣性**逐步推导最优解。

5.4.1 运动轨迹及约束条件更新

三枚烟幕干扰弹分别由三个不同位置出发的无人机所携带并投放和引爆，故干扰弹投放和引爆的坐标范围必须在各自所属无人机的飞行线平面。因此，可首先根据无人机坐标以及飞行角度 θ_1 、 θ_2 、 θ_3 计算出三个无人机 FY1、FY2、FY3 各自的飞行平面：

$$\begin{cases} FY1 : y = \tan \theta_1 (x - 17800) \\ FY2 : y = \tan \theta_2 (x - 12000) + 1400 \\ FY3 : y = \tan \theta_3 (x - 6000) - 3000 \end{cases} \quad (42)$$

由此，可得三枚干扰弹投放与引爆的坐标范围约束，即必须位于无人机飞行平面：

$$\begin{cases} (x_{11}, y_{11}, z_{11}), (x'_{11}, y'_{11}, z'_{11}) \in FY1 \\ (x_{21}, y_{21}, z_{21}), (x'_{21}, y'_{21}, z'_{21}) \in FY2 \\ (x_{31}, y_{31}, z_{31}), (x'_{31}, y'_{31}, z'_{31}) \in FY3 \end{cases} \quad (43)$$

同时，针对无人机、烟幕干扰弹、烟幕云团的坐标变化方程，可参考上文模型并根据无人机飞行方向 θ_1 、 θ_2 、 θ_3 依次给出即可。此处，本文选择给出烟幕干扰弹平抛运动过程对应的坐标变化方程以作调整示例，其余运动方程将在后文模型总结中详细给出。

$$\begin{cases} z_g = z_{g0} - \frac{1}{2}g(t - t_{g0})^2 \\ x_g = x_{g0} + v_w(t - t_{g0}) \cos \theta_i \\ y_g = v_w t \sin \theta_i, i = \{1, 2, 3\} \end{cases} \quad (44)$$

5.4.2 云团相对位置及遮挡情况分析

在上文多烟幕云团部分重叠组合分析的基础上,在此处对问题四五中两烟幕云团可能存在的三种相对位置关系(相离、组合、重叠)展开分析说明,首先给出在 z 轴方向剖面上三者的相对形态示意图如下。其中,红色虚线表示射影曲线外轮廓;蓝色虚线表示穿过烟幕云团即被遮挡的射影曲线;绿色圆球表示烟幕云团有效遮挡范围。

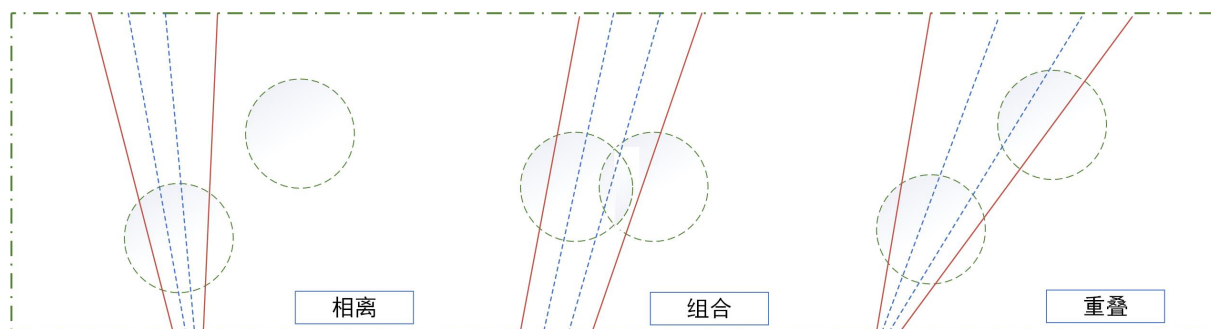


图 15 两云团间可能的相对形态(z 轴方向剖面)示意图

- 相离: 一个烟幕云团即可实现真目标的有效全遮挡。
- 组合: 两个烟幕云团协同为目标圆柱体提供有效全遮蔽。
- 重叠: 一个烟幕云团已经可以有效全遮挡的情况下,另一个烟幕云团也对目标圆柱体的部分提供遮蔽,意味着重叠的烟雾云团的一部分无效作用浪费了其有效时间。

综上所述,相对位置**重叠**形态为对遮蔽效果不太友好的一种情况,因此在策略方面,要让结果**尽量少出现**这种类型情况以充分利用烟幕云团达到更长久的遮蔽效果。

5.4.3 决策机理分析

根据模型直接计算不仅计算量大,而且不加以策略限制进行求解也容易在求解中导致陷入局部最优的情况,因此本文选择在计算求解前制定一个初步的求解策略限制。

首先,对问题四的情况分析可知,三个无人机中单个无人机的投弹引爆过程与问题二的情形一致,因此三个无人机的投弹问题可视为三个问题二情形的组合;同时,多个烟幕云团之间存在三种形态关系,综上,问题四经分析可视为“三个问题二情形”与“烟幕云团间形态关系”的组合问题。此外,对于求解中关于烟幕云团间的关系选择需遵循一定的策略:根据上文所述的三种烟幕云团关系可知,**有效遮挡利用率最低的重叠情形应尽量规避**,而其余两种相对位置分布情况可以允许发生。

据此,本问初步求解策略设定如下:首先依次求解三个无人机在问题二情形下的最优引爆点位置,以及对应的投弹时间、无人机飞行方向及速度等参数;由此得到的三个引爆点位置为初步策略下的初始较优解,但大概率并非全局最优解。随后基于该初始较优解开展更全面的算法优化与筛选,且在优化过程中需尽量避免烟幕云团重叠的情况出现,以助于最终得到全局最优解及相应的投弹策略。

5.4.4 模型总结

由于在问题四的背景下，烟幕云团会存在重叠与组合的位置状态。故在计算有效遮挡时间时，会重复计算重叠部分与组合体公共部分的遮挡时间，需在原目标函数 $\int I_{[\Delta]} I_{[t]} dt$ 的基础上减去重复计算值 $\int I_{[\Delta]} I_{[t]} dt_{\text{重叠}}$ 与 $\int I_{[\Delta]} I_{[t]} dt_{\text{组合}}$ 。

联立适应性调整后的目标函数与运动方程，即可得到**多机单次投弹策略优化模型**：

$$\begin{aligned} & \max \left(\int I_{[\Delta]} I_{[t]} dt - \int I_{[\Delta]} I_{[t]} dt_{\text{重叠}} - \int I_{[\Delta]} I_{[t]} dt_{\text{组合}} \right) \quad (45) \\ & s.t. \begin{cases} I_{[\Delta]} = \begin{cases} 1, \text{任意点满足连线} \Delta \geq 0 \\ 0, \text{任意点满足连线} \Delta < 0 \end{cases} & I_{[t]} = \begin{cases} 1, t - t_b \leq 20 \\ 0, t - t_0 > 20 \end{cases} \\ \begin{cases} at^2 + bt + c = 0 \\ \Delta = b^2 - 4ac \end{cases} & \begin{cases} a = 1 & b = 2[n_1(x_d - x_b) + n_2(y_d - y_b) + n_3(z_d - z_b)] \\ c = [(x_d - x_b)^2 + (y_d - y_b)^2 + (z_d - z_b)^2 - R^2] \end{cases} \\ (x - x_b)^2 + (y - y_b)^2 + (z - z_b)^2 = R^2 & \vec{n} = \frac{\vec{m}}{\|\vec{m}\|} = (n_1, n_2, n_3) \\ \vec{L}(\rho) = \vec{d} + \vec{n}\rho & \vec{d} = (x_d, y_d, z_d) \quad \vec{m} = (x_1 - x_d, y_1 - y_d, z_1 - z_d) \\ \begin{cases} z_d = z_{d0} - v_d t \sin \theta \\ x_d = x_{d0} - v_d t \cos \theta \\ y_d = 0 \quad x_d \in [0, x_{d0}] \end{cases} & \begin{cases} z_w = z_{w0} = 1800 \\ x_w = x_{w0} + v_w t \cos \alpha \\ y_w = v_w t \sin \alpha \end{cases} & \begin{cases} x_{b,k} = x_{b0,k} \\ y_{b,k} = y_{b0,k} \\ z_{b,k} = z_{b0,k} - v_{b,k}(t - t_{b,k}) \end{cases} \\ \begin{cases} z_{g,k} = z_{g0,k} - \frac{1}{2}g(t - t_{g0,k})^2 \\ x_{g,k} = x_{g0,k} + v_w(t - t_{g0,k}) \cos \alpha \\ y_{g,k} = v_w t \sin \alpha \end{cases} & \begin{cases} (x_{1i}, y_{1i}, z_{1i}) \in y = \tan \alpha (x - 17800) \\ (x'_{1i}, y'_{1i}, z'_{1i}) \in y = \tan \alpha (x - 17800) \\ z_{1i} > 0, z'_{1i} > 0, (i = 1, 2, 3) \end{cases} \\ \begin{cases} FY1 : y = \tan \theta_1 (x - 17800) \\ FY2 : y = \tan \theta_2 (x - 12000) + 1400 \\ FY3 : y = \tan \theta_3 (x - 6000) - 3000 \end{cases} & \begin{cases} (x_{11}, y_{11}, z_{11}), (x'_{11}, y'_{11}, z'_{11}) \in FY1 \\ (x_{21}, y_{21}, z_{21}), (x'_{21}, y'_{21}, z'_{21}) \in FY2 \\ (x_{31}, y_{31}, z_{31}), (x'_{31}, y'_{31}, z'_{31}) \in FY3 \end{cases} \\ \alpha \in [0, 360^\circ] \quad v_w \in [70m/s, 140m/s] \quad k \in \{1, 2, 3\} \\ \|(x_{1i}, y_{1i}, z_{1i}) - (x_w, y_w, z_w)\| < \|(x_{1j}, y_{1j}, z_{1j}) - (x_w, y_w, z_w)\|, (i = 1, 2, 3; i < j) \\ \|(x_{1i}, y_{1i}, z_{1i}) - (x_{1(i+1)}, y_{1(i+1)}, z_{1(i+1)})\| \geq v_w \cdot 1, (i = 1, 2, 3) \\ \Omega_i(t) = [A_{j1}(t) + A'_{i1}(t)] I_{(i1)(j1)}(t) + A_{i1}(t) [1 - I_{(i1)(j1)}(t)], (i, j = 1, 2, 3, i \neq j) \end{cases} \end{aligned}$$

其中，为便于后续模型五中的表示，本文选择将该 s.t 后的约束条件统称为 **Model4**。

5.4.5 策略设计及模型求解

■ 单对单策略

本问针对问题四提出了一种“单对单策略”进行解的初步运算，策略基本思想如下：

首先，观察到问题四与问题二在本质上具有相似性：问题二中，单架无人机对单枚导弹进行干扰时，可以通过优化干扰弹投放参数来最大化遮蔽时长；问题四中，有三架无人机同时参与干扰单枚导弹，但各无人机的速度与投放行为可以认为是相互独立的。

基于这一原则，可将问题四拆解为三个独立的子问题，每个子问题对应一架无人机对导弹的干扰。这一步得到的解称为“初解”，即每架无人机在单独作用下的最优方案。

得到三个初解后，可得到三组遮蔽时间区间和对应的最大遮蔽时长。在此基础上，考虑三架无人机投放的烟雾弹叠加所产生的遮蔽问题，会导致实际有效遮蔽时间不等于各自遮蔽时间的简单累加，优化该遮蔽情况进而逐步获得最终的整体最优解。该方法不仅可简化问题求解的复杂度，而且可为实际多无人机协同干扰提供初步实施方案。

■ 多子群 + 蒙特卡洛 + GA-NM 的遮挡优化方法

在本问中，多无人机的引入会导致原本解空间的向量可变参数数量达到 12 个，并且无人机速度与时间的搜索空间过大，使用上文的 GA-NM 融合算法求解时容易陷入局部最优解。因此，本问首先采用“单对单策略”，将问题拆分为三个子问题。同时，为了尽可能避免出现局部最优解，算法设计了一种“蒙特卡洛采样 + 多子群遗传算法-Nelder-Mead 局部优化”的混合智能优化方法，具体操作详见算法思路部分 [6][7]。

此时，优化变量与目标函数的数学表达式如下所示。其中，优化变量为干扰弹的速度分量、释放时间与起爆时间组成的变量组，目标函数为最大化遮挡函数。

$$\begin{cases} \text{优化变量: } x = (v_x, v_y, t_1, t_2) \\ \text{目标函数: } \max f(x) = T_{\text{occlusion}}(v_x, v_y, t_1, t_2) \\ 70 \leq \sqrt{v_x^2 + v_y^2} \leq 140 \quad 0 \leq t_1 \leq t_2 \leq 20 \end{cases} \quad (46)$$

■ 算法思路

Step1. 单对单策略求初解

首先借助“单对单策略”的思想将初始问题拆分为三个子问题。通过分别对一个无人机求初解，可以初步对解向量的范围进行预判，以避免了后续搜索的盲目性。

Step2. 蒙特卡洛采样

在解空间内对初解周围进行大规模采样，随机生成 5000 个随机解，筛选出前 50 个较优解作为遗传算法的初始种群，避免陷入局部最优。

Step3. 多子群并行进化

将总群体划分为四个子群，每个子群独立运行进化算法，探索搜索空间的不同区域。这种并行机制可以有效增加种群的多样性，避免陷入同一局部最优。

Step4. 遗传算法

在多子群并行优化的基础上，采用遗传算法对蒙特卡洛随机生成的较优解进行模拟，计算出的情况，为了确保局部精度高，进一步进行局部优化处理。

Step5.Nelder-Mead 算法局部优化

在遗传算法得到的最优解基础上，用单纯形局部优化方法对解进一步优化处理 [7]。

■ 结果展示与分析

通过执行上述策略及算法思想，运行代码后求解出问题四的结果整理如下所示：

表 3 多机单次投策策略优化模型求解结果一

无人机编号	无人机方向 (飞行角度 α)	无人机速度 (m/s)	各自有效遮蔽时长 (s)	整体有效遮蔽时长 (s)
FY1	178.57	117.1486	4.5154	10.4172
FY2	245.21	48.7629	4.5817	
FY3	66.82	124.3448	2.1026	

其中，整体有效遮蔽时长是指在考虑“相离、组合、重叠”三类云团相对形态分布后，计算拟合出的最终整体有效遮蔽时长。若未考虑”组合”与”重叠”的情况，计算结果一定高于本文的结果，但本文在考虑形态分布后再计算结果，该结果的逻辑合理性与遮准确性一定更高。

表 4 多机单次投策策略优化模型求解结果二

无人机编号	各自有效遮蔽时长结果 (s)	投放坐标:x	y	z	起爆坐标:x	y	z
FY1	4.5154	17790.34	0.24	1800.00	17417.41	9.57	1750.31
FY2	4.5817	11546.17	417.59	1400.00	11458.85	228.57	1310.66
FY3	2.1026	7567.84	661.37	700.00	7736.53	1055.3	641.81

由此可知，多子群与蒙特卡洛结合的策略，在运算初期有效生成了较优初始解，能有效避免算法陷入局部最优；随后通过遗传算法实现对较优解空间的全局粗搜索，再经 Nelder-Mead (NM) 单纯形法进一步提升解的精度，最终总有效遮蔽时间稳定在 10.4172 秒——这表明在多无人机投放策略下，总遮蔽时间得到进一步提升，但仍存在 0.7825 秒的遮蔽重叠时间。结合问题四的分析可知，当前“单对单”投放策略仍有优化空间：后序可从时间与空间双维度优化策略，通过调整各无人机的投放时间与飞行速度，使三架无人机的遮蔽区间在时空上合理错开或适度叠加，进而实现遮蔽时间的优化。

5.5 问题五：多机多次投弹策略优化模型

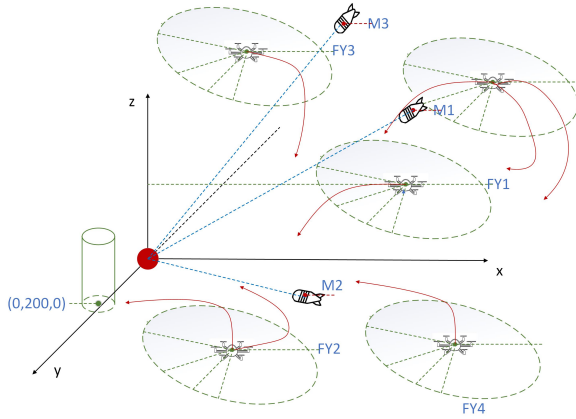


图 16 问题五求解情形示意图

本问增加无人机的数量至 5 驾、增加来袭导弹数量至 3 枚、且约束每驾无人机的投弹数量不超过三枚，属于多机多次投弹策略优化模型。此时问题情况较为复杂，决策灵活度较高，下文会仔细分析一些新情况并对问题进行拆解以确立初步求解策略。

具体示意情形可如左图所示，其中红色曲线表示可多次投弹、绿色椭圆曲线表示无人机飞行方向的可选择性。

5.5.1 运动过程约束情况分析

首先，无人机的飞行偏角 θ_i 可确定其各自的飞行轨迹并限制其投弹的空间范围：

$$(x_{ij}, y_{ij}, z_{ij}) \in \pi_i, (x'_{ij}, y'_{ij}, z'_{ij}) \in \pi_i, i = \{1, 2, 3, 4, 5\}, j = \{1, 2, 3\} \quad (47)$$

其中， π_i 表示无人机 FY_i 飞行路线所确定的投弹区域平面 ($i \in \{1, 2, 3, 4, 5\}$)。

其次，根据题干要求可知每个无人机可选择投放的烟幕干扰弹数量不得超过 3 枚：

$$\begin{cases} I_{[ij]} = \begin{cases} 1, \text{无人机 } FY_i \text{ 的第 } j \text{ 颗烟幕弹被投放} \\ 0, \text{无人机 } FY_i \text{ 的第 } j \text{ 颗烟幕弹未被投放} \end{cases} \\ \sum_{j=1}^3 I_{[ij]} \leq 3, i = \{1, 2, 3, 4, 5\} \end{cases} \quad (48)$$

其中， $I_{[ij]}$ 是对上文示性函数调整所得，用于刻画无人机 FY_i 的干扰弹投放情况。

同时，同一无人机的相邻投弹时间间距不得小于 $1s$ ，即来源于同一无人机的相邻两烟幕干扰弹投放点距离不小于 $v_{wi} \cdot 1$ ，具体以不等式的数学形式刻画如下所示。

$$\|(x_{ij}, y_{ij}, z_{ij}) - (x_{i(j+1)}, y_{i(j+1)}, z_{i(j+1)})\| \geq v_{wi} \cdot 1, (i = \{1, 2, 3, 4, 5\}) \quad (49)$$

此外，其余运动规律 (如平抛运动) 与前文所述基本一致，会在模型总结部分给出。

5.5.2 决策机理分析

问题五中引入三个来袭导弹，除已有的两烟幕云团间三种形态关系外，该问题还新增“单个烟幕云团同时对多枚导弹产生干扰”的情况：具体表现为某一时刻下，三个导

弹各自对应真目标的射影曲线上的点，与对应导弹的连线均穿过同一云团，这一情况显著提升了该类烟幕云团的遮蔽利用率，基于该**提升遮蔽利用率**的思想可提出如下决策。

◦ 高度优先级策略

由于导弹以匀速朝向假目标原点 O 飞行，其垂直高度会均匀降低。基于此特性，投弹策略设定如下：当导弹高度较高时，高度较高的无人机应优先对其投弹干扰；当导弹飞行至较低高度时，再由高度较低的无人机对其投弹干扰。该投弹策略能让烟幕云团及时靠近导弹，进而增大导弹视野盲区的张角，更高效地遮蔽目标圆柱体。

◦ 重心初步代替策略

三个导弹的飞行轨迹固定，均为从初始位置指向原点的直线；且速度恒定为 300 m/s，因此三者连线构成的三角形 $\triangle M_1 M_2 M_3$ 在运动过程中形状保持不变，仅大小逐渐缩小。同时由于该三角形的重心与三个顶点的距离相同，重心遮蔽情况与三个顶点的差异不会很大，可初步考虑采用该三角形的重心 M 来代替三个导弹考虑遮蔽情况：

$$M = \left(\frac{\sum_{i=1}^3 M_{ix}}{3}, \frac{\sum_{i=1}^3 M_{iy}}{3}, \frac{\sum_{i=1}^3 M_{iz}}{3} \right)$$

据此求解投放策略，再根据三个导弹的实际位置来优化，即可得到最优投弹策略。

◦ 烟幕云团相对位置分配策略

需尽量避免两烟幕云团重叠的出现 (以免降低遮蔽利用率)，但两烟幕云团的组合情况是可接受的。此外，能同时对多枚导弹形成较大范围干扰的烟幕云团，应尽可能保留。

5.5.3 模型总结

在模型四的基础上整合施加部分新的约束条件，得出**多机多次投弹策略优化模型**：

$$\begin{aligned} & \max \left\{ \int I_{[\Delta]}(t) I_{[t]} dt - \int dt_{\text{重叠}} - \int dt_{\text{组合}} \right\} \quad (50) \\ & s.t \left\{ \begin{array}{l} \text{Model4: 指代模型四中的约束条件} \\ I_{[\Delta]}(t) = \begin{cases} 1, \text{若} t \text{时刻三个导弹全被遮蔽} \\ 0, \text{若} t \text{时刻存在导弹未被完全遮蔽} \end{cases} \\ (x_{ij}, y_{ij}, z_{ij}) \in \pi_i, (x'_{ij}, y'_{ij}, z'_{ij}) \in \pi_i, i = \{1, 2, 3, 4, 5\}, j = \{1, 2, 3\} \\ I_{[ij]} = \begin{cases} 1, \text{无人机FY} i \text{的第} j \text{颗烟幕弹被投放} \\ 0, \text{无人机FY} i \text{的第} j \text{颗烟幕弹未被投放} \end{cases} \\ \sum_{j=1}^3 I_{[ij]} \leq 3, i = \{1, 2, 3, 4, 5\} \\ \|(x_{ij}, y_{ij}, z_{ij}) - (x_{i(j+1)}, y_{i(j+1)}, z_{i(j+1)})\| \geq v_{wi} \cdot 1, (i = \{1, 2, 3, 4, 5\}) \end{array} \right. \end{aligned}$$

5.5.4 最优投弹策略求解

首先，通过 **K-Means 聚类 + 单对单 / z 值优先 / 空间邻近策略**分配无人机任务：

表 5 无人机任务分配步骤

步骤	描述
Step 1	问题初始化与参数设置 设定导弹初始位置 M_{i0} 和速度向量 v_i ($i = 1, 2, 3$)，确定时间区间 $[0, T_{\max}]$ 和离散化步长 Δt （建议取 0.5 秒）。
Step 2	导弹轨迹聚类分析 使用 K-Means 算法将 3 枚导弹的预测落点聚为 3 类，每类对应一个空间簇；根据聚类结果，将 5 架无人机分配至最近导弹簇，确定每架无人机的主要干扰目标。
Step 3	任务分配策略应用 应用三种基础策略细化任务分配： • 单对单策略：为每架无人机指定主要干扰目标； • z 值优先策略：按无人机高度分配任务，高海拔无人机优先； • 空间邻近策略：依据欧氏距离将无人机分配至最近导弹。

其次，**导弹轨迹反演和 GA+NM 混合优化算法**精确确定烟幕投放时刻与空间位置，从而最大化三个导弹同时遮蔽的有效时间，保证任务的高效执行，具体步骤如下 [7]：

表 6 多无人机协同干扰算法步骤

步骤	描述
Step 4	导弹轨迹反演计算 从末端时刻 $T_{\max}$ 开始逆向遍历每一时刻 t_b ，计算导弹位置： $X_i(t_b) = M_i(t_b) = M_{i0} + v_i t_b, \quad i = \{1, 2, 3\}$
Step 5	视锥建模与采样 在每个时刻 t_b 下，建立三导弹对目标的视锥模型。通过在圆柱体表面均匀采样 100-200 个判定点，评估可能的遮蔽区域。
Step 6	烟雾球优化求解 对于每个候选时刻 t_b ，采用两阶段优化方法确定最优烟雾球参数： • 全局搜索阶段：使用遗传算法 (GA) 确定球心 C 和半径 R 的粗略最优值 • 局部优化阶段：使用 Nelder-Mead 算法对 GA 结果进行精细优化 • 适应度函数为三导弹同时遮蔽的有效时长： $T_{\text{cover}}(C) = \sum_t \mathbf{1}_{\text{遮蔽有效}} \cdot \Delta t$

对所有候选时刻 t_b 重复 Step 4-6，选取全局最优解，得到最佳投弹时刻与球心位置：

$$(t_b^*, C_b^*) = \arg \max_{t_b} T_{\text{cover}}(C_b)$$

根据最优决策结果 (t_b^*, C_b^*) ，协调 5 架无人机的飞行路径和投弹时序，确保在正确时刻投放干扰弹到目标位置，实现最大遮蔽效果。最终的模型求解结果整理如下所示：

表 7 无人机飞行策略核心结果

无人机编号	FY1	FY2	FY3	FY4	FY5
运动方向 α	177.1222	133.8763	150.4612	267.1222	88.6729
运动速度 (m/s)	128.9502	58.9634	123.6364	115.3913	108.5984
干扰的导弹编号	M1	M2	{M1,M2,M3}	M1	M3
整体有效干扰时长 (s)	12.4828				

其中，整体有效遮蔽时长 12.4828s 为本题的核心计算结果；干扰的导弹编号是本文的求解结果创新点，即求解计算出了每个无人机所负责遮蔽的导弹编号。

其他参数信息如投弹点坐标、起爆点坐标等，本文在正文中按照无人机编号顺序依次给出如下所示，至此附件中三个 excel 表格的所有信息数据均在正文中有所体现：

表 8 无人机 FY1 具体投弹策略

干扰弹编号	有效遮蔽时长 (s)	投放坐标:x	y	z	起爆坐标:x	y	z
1	3.3327	17789.2988	0.1946	1800	17460.2221	6.1773	1748.9516
2	4.2313	17564.2710	11.8497	1800	17255.0637	27.3931	1771.7546
3	3.3380	17799.4422	0.0280	1800	17370.1090	21.6100	1745.5451

表 9 无人机 FY2 具体投弹策略

干扰弹编号	有效遮蔽时长 (s)	投放坐标:x	y	z	起爆坐标:x	y	z
1	4.9995	11975.2313	425.7598	1400	11795.6807	612.4948	1305.4181
2	3.8046	11904.8937	498.9119	1400	11844.5136	561.7079	1389.3040
3	5.1222	11814.7330	592.6802	1400	11790.6671	617.7090	1398.3008

表 10 无人机 FY3 具体投弹策略

干扰弹编号	有效遮蔽时长 (s)	投放坐标:x	y	z	起爆坐标:x	y	z
1	1.4276	-2167.6134	4659.9522	700	107.5664	139.4336	7.1414
2	1.4276	-2275.1798	4720.9065	700	0	200.0000	7.1414
3	1.4276	-2382.7462	4781.8608	700	-107.5566	260.5664	7.1414

表 11 无人机 FY4 具体投弹策略

干扰弹编号	有效遮蔽时长 (s)	投放坐标:x	y	z	起爆坐标:x	y	z
1	2.9431	10876.1634	52.3618	1800	10486.1684	44.3478	1151.2861
2	2.4231	10932.1454	38.4884	1800	10652.1564	28.4861	1175.1685
3	2.8461	11361.6512	46.4894	1800	10761.6482	34.4862	1138.1864

表 12 无人机 FY5 具体投弹策略

干扰弹编号	有效遮蔽时长 (s)	投放坐标:x	y	z	起爆坐标:x	y	z
1	3.5641	15924.4517	-537.5246	1300	17345.4518	-486.5651	1208.4828
2	2.5618	16245.6495	-487.5466	1300	17245.6489	-449.5576	1266.2543
3	3.8415	16843.4613	-512.4665	1300	17246.4613	-427.1565	1328.3541

六、模型的评价与推广

6.1 模型的优点

- (1) 策略机理分析完整：针对同一模型采用不同策略对比求解，结果精度更高。
- (2) 算法创新：采取融合性优化算法求解，而非直接在域上使用正向搜索遍历求解。
- (3) 结果可靠性高：利用鲁棒性、灵敏度、仿真模拟等方法检验求解结果的可靠性。
- (4) 可视化效果丰富：在模型中多处以透视图等形式展现几何关系，直观性强。

6.2 模型的缺点

- (1) 模型假设与现实实际情况仍存在较大差异，模型的实际应用性可能存在不足。
- (2) 模型对题干给定信息的依赖性较强，若该信息改变则模型适用性会受到影响。

6.3 模型的推广

(1) 反演计算-正向搜索验证

该算法策略思想可应用到其他问题领域，在求解模型时，若正向直接搜索或推理求解太过复杂不易操作，可以利用反演的思想反向推导计算，最后用正向搜索来结果比对验证，达到双向对比求解的效果，以避免陷入局部最优解。

(2) 结果可靠性分析

本文所采用的鲁棒性分析、灵敏度分析、仿真模拟等结果可靠性分析方法可应用至其他模型中，通过结果可靠性分析可使得模型体系更加完整可靠。

七、参考文献

- [1] 杜得志. 平抛运动中的三类落点问题分类与分析 [J]. 数理天地 (高中版), 2025, (02): 4-5.
- [2] 吴佐慧. 射影几何在高中圆锥曲线问题中的应用 [J]. 数学通讯, 2020, (14): 41-43+46.
- [3] 雍龙泉, 刘三阳. 单纯形法的复杂性与计算效率 [J]. 高等数学研究, 2024, 27(03): 50-52+55.
- [4] 曾佳, 申功璋. 一种无人机自主变步长航迹规划方法 [J]. 弹箭与制导学报, 2008, 28(06): 21-24.
- [5] 金芬. 遗传算法在函数优化中的应用研究 [D]. 苏州大学, 2008.
- [6] 徐磊. 基于遗传算法的多目标优化问题的研究与应用 [D]. 中南大学, 2007.
- [7] CHAT-GPT5, OPEN AI, 2025-09-05

附录 A 支撑材料文件附录

附录 A：支撑材料的文件列表

AI 工具使用详情.pdf

问题一文件夹：在问题一求解中使用到的所有 Python 代码

问题二文件夹：在问题二求解中使用到的所有 Python 代码

问题三文件夹：在问题三求解中使用到的所有 Python 代码

问题四文件夹：在问题四求解中使用到的所有 Python 代码

问题五文件夹：在问题五求解中使用到的所有 Python 代码

result1.xlsx 文件：问题三最终求解结果

result2.xlsx 文件：问题四最终求解结果

result3.xlsx 文件：问题五最终求解结果

附录 B 代码目录

附录 B：代码目录

问题一所有相关求解代码 P35-P49

问题二所有相关求解代码 P50-P64

问题三所有相关求解代码 P65-P74

问题四所有相关求解代码 P75-P79

问题五所有相关求解代码 P80-P87

Listing 1: 问题一 变时间步长求解

```
import math
import numpy as np

M1_dangqian_weizhi = (20000, 0, 2000)
yanwu_dangqian_weizhi = (17800, 0, 1800)
jiaodu = math.atan(2000/20000)
sudu_M = (-300*math.cos(jiaodu), 0, -300*math.sin(jiaodu))
sudu_yanwu = (-120, 0, 0)
sudu_yanwu2 = (0, 0, -3)
shijian1 = 1.5
shijian2 = 3.6
g = 9.8
yuanzhu_zhongxin = (0, 200, 0)
yuanzhu_r = 7
yuanzhu_h = 10

def weizhi_M1(t):
    x = M1_dangqian_weizhi[0] + (t) * sudu_M[0]
    y = M1_dangqian_weizhi[1] + (t) * sudu_M[1]
    z = M1_dangqian_weizhi[2] + (t) * sudu_M[2]
    return (x, y, z)

def weizhi_yanwu(t):
    if t >= 0 and t < shijian1:
        x = yanwu_dangqian_weizhi[0] + (t) * sudu_yanwu[0]
        y = yanwu_dangqian_weizhi[1] + (t) * sudu_yanwu[1]
        z = yanwu_dangqian_weizhi[2] + (t) * sudu_yanwu[2]
        return (x, y, z)
    elif t >= shijian1 and t <= shijian1 + shijian2:
        x = yanwu_dangqian_weizhi[0] + shijian1 * sudu_yanwu[0] + (t - shijian1) * sudu_yanwu[0]
        y = yanwu_dangqian_weizhi[1] + shijian1 * sudu_yanwu[1] + (t - shijian1) * sudu_yanwu[1]
        z = yanwu_dangqian_weizhi[2] + shijian1 * sudu_yanwu[2] + (t - shijian1) * sudu_yanwu[2]
        - g * (t - shijian1) * (t - shijian1) / 2
        return (x, y, z)
    else:
        x = yanwu_dangqian_weizhi[0] + shijian1 * sudu_yanwu[0] + shijian2 * sudu_yanwu[0] + (t
            - (shijian1 + shijian2)) * sudu_yanwu2[0]
        y = yanwu_dangqian_weizhi[1] + shijian1 * sudu_yanwu[1] + shijian2 * sudu_yanwu[1] + (t
            - (shijian1 + shijian2)) * sudu_yanwu2[1]
        z = yanwu_dangqian_weizhi[2] + shijian1 * sudu_yanwu[2] + shijian2 * sudu_yanwu[2] + (t
            - (shijian1 + shijian2)) * sudu_yanwu2[2] - g * shijian2 * shijian2 / 2
        return (x, y, z)

def jiaodian_qiuti(P, Q, C, r):
    PQ = Q - P
```

```

a = np.dot(PQ, PQ)
b = 2 * np.dot(PQ, P - C)
c = np.dot(P - C, P - C) - r**2
D = b*b - 4*a*c
if D < 0:
    return False
sqrtD = math.sqrt(D)
s1 = (-b - sqrtD) / (2*a)
s2 = (-b + sqrtD) / (2*a)
return (0 <= s1 <= 1) or (0 <= s2 <= 1)

def yuanzhu_bianjie_dian(n_theta=64, n_z=4):
    dian = []
    for k in range(n_z+1):
        z = k * yuanzhu_h / n_z
        for i in range(n_theta):
            angle = 2 * math.pi * i / n_theta
            x = yuanzhu_zhongxin[0] + yuanzhu_r * math.cos(angle)
            y = yuanzhu_zhongxin[1] + yuanzhu_r * math.sin(angle)
            dian.append((x, y, z))
    return dian

bianjie_dian = yuanzhu_bianjie_dian()

T = 25.1
qiuti_r = 10
dt_min = 0.001
dt_max = 0.02
k = 1e-5
youxiao_shijian = []
t = 0.0
while t <= T:
    Q = np.array(weizhi_M1(t))
    C = np.array(weizhi_yanwu(t))
    ok = True
    for bp in bianjie_dian:
        P = np.array(bp)
        if not jiaodian_qiuti(P, Q, C, qiuti_r):
            ok = False
            break
    if ok:
        youxiao_shijian.append(round(t, 4))
    d = np.linalg.norm(Q - np.array(yuanzhu_zhongxin))
    dt = max(dt_min, min(dt_max, k * d))
    t += dt

if youxiao_shijian:

```

```
t_start = youxiao_shijian[0]
t_end = youxiao_shijian[-1]
chixu = round(t_end - t_start, 4)
print(f"球体完全覆盖圆柱的开始时间: {t_start} s")
print(f"球体完全覆盖圆柱的结束时间: {t_end} s")
print(f"总持续时间: {chixu} s")
else:
    print("25.1秒内没有出现球体完全覆盖圆柱的情况")
```

Listing 2: 采样法鲁棒性分析

```

import math
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import cm
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

m1_dangqian = (20000, 0, 2000)
yan_dangqian = (17800, 0, 1800)
jiaodu = math.atan(2000/20000)
sudu_m = (-300*math.cos(jiaodu), 0, -300*math.sin(jiaodu))
sudu_yan = (-120, 0, 0)
sudu_yan2 = (0, 0, -3)
g = 9.8
zhu_zhongxin = (0, 200, 0)
zhu_banjing = 7
zhu_gaodu = 10
t_zong = 25.1
qiu_banjing = 10

def weizhi_m1(t):
    x = m1_dangqian[0] + t * sudu_m[0]
    y = m1_dangqian[1] + t * sudu_m[1]
    z = m1_dangqian[2] + t * sudu_m[2]
    return np.array([x, y, z])

def weizhi_yan(t):
    if t < 1.5:
        x = yan_dangqian[0] + t*sudu_yan[0]
        y = yan_dangqian[1] + t*sudu_yan[1]
        z = yan_dangqian[2] + t*sudu_yan[2]
    elif t <= 5.1:
        x = yan_dangqian[0] + 1.5*sudu_yan[0] + (t-1.5)*sudu_yan[0]
        y = yan_dangqian[1] + 1.5*sudu_yan[1] + (t-1.5)*sudu_yan[1]
        z = yan_dangqian[2] + 1.5*sudu_yan[2] + (t-1.5)*sudu_yan[2] - 0.5*g*(t-1.5)**2
    else:
        x = yan_dangqian[0] + 1.5*sudu_yan[0] + 3.6*sudu_yan[0] + (t-5.1)*sudu_yan2[0]
        y = yan_dangqian[1] + 1.5*sudu_yan[1] + 3.6*sudu_yan[1] + (t-5.1)*sudu_yan2[1]
        z = yan_dangqian[2] + 1.5*sudu_yan[2] + 3.6*sudu_yan[2] + (t-5.1)*sudu_yan2[2] -
            0.5*g*3.6**2
    return np.array([x, y, z])

def jiaodian_qiu(P, Q, C, r):
    PQ = Q - P
    a = np.dot(PQ, PQ)

```

```

b = 2 * np.dot(PQ, P - C)
c = np.dot(P - C, P - C) - r**2
D = b*b - 4*a*c
if D < 0:
    return False
sqrtD = math.sqrt(D)
s1 = (-b - sqrtD) / (2*a)
s2 = (-b + sqrtD) / (2*a)
return (0 <= s1 <= 1) or (0 <= s2 <= 1)

def zhu_bianjie_dian(n_jiaodu, n_z=8, fanrao=0.0):
    dian = []
    for k in range(n_z+1):
        z = k * zhu_gaodu / n_z
        for i in range(n_jiaodu):
            jiaodu = 2 * math.pi * i / n_jiaodu
            delta = np.random.uniform(-fanrao, fanrao)
            x = zhu_zhongxin[0] + zhu_banjing * math.cos(jiaodu + delta)
            y = zhu_zhongxin[1] + zhu_banjing * math.sin(jiaodu + delta)
            dian.append(np.array([x, y, z]))
    return dian

def jisuan_shichang(n_jiaodu, fanrao):
    bianjie_dian = zhu_bianjie_dian(n_jiaodu, fanrao=fanrao)
    shijian = []
    t = 0.0
    dt_min = 0.01
    dt_max = 0.5
    k = 1e-6
    while t <= t_zong:
        Q = weizhi_m1(t)
        C = weizhi_yan(t)
        ok = all(jiaodian_qiu(P, Q, C, qiu_banjing) for P in bianjie_dian)
        if ok:
            shijian.append(t)
            d = np.linalg.norm(Q - np.array(zhu_zhongxin))
            dt = max(dt_min, min(dt_max, k * d))
            t += dt
    if shijian:
        return shijian[-1] - shijian[0]
    else:
        return 0.0

jiaodu_list = [8, 16, 32, 64, 128, 256, 512]
fanrao_list = np.linspace(0, 2, 10)

shichang = np.zeros((len(fanrao_list), len(jiaodu_list)))

```

```

for i, f in enumerate(fanrao_list):
    for j, n in enumerate(jiaodu_list):
        shichang[i,j] = jisuan_shichang(n, f)
        print(f"fanrao={f:.3f}, jiaodu={n}, zhebi={shichang[i,j]:.4f}s")

fig = plt.figure(figsize=(12,6))
ax = fig.add_subplot(111, projection='3d')
_x = np.arange(len(jiaodu_list))
_y = np.arange(len(fanrao_list))
_xx, _yy = np.meshgrid(_x, _y)
x, y = _xx.ravel(), _yy.ravel()
top = shichang.ravel()
bottom = np.zeros_like(top)
kuan = shen = 0.6
ax.bar3d(x, y, bottom, kuan, shen, top, shade=True, color=cm.viridis(top/max(top)))
ax.set_xticks(_x + kuan/2)
ax.set_xticklabels(jiaodu_list)
ax.set_xlabel("zhu caixiangdian shuliang", labelpad=10)
ax.set_yticks(_y + shen/2)
ax.set_yticklabels([f"{f:.3f}" for f in fanrao_list])
ax.set_ylabel("caixiangdian fanrao (m)", labelpad=10)
ax.set_zlabel("zhebi shijian (s)", labelpad=10)
ax.set_title("caixiangdian shuliang yu fanrao dui zhebi shijian yingxiang", pad=20)
plt.show()

```


Listing 3: 问题一方法二对比求解

```
import math
import numpy as np

g = 9.8
banjing = 10
zhongxin = np.array([0, 200, 0])
zhu_banjing = 7
zhu_gaodu = 10
jiaodu = math.atan(2000/20000)
sudu_dandao = (-300*math.cos(jiaodu), 0, -300*math.sin(jiaodu))
sudu_yan = (-120, 0, 0)
sudu_yan2 = (0, 0, -3)
sudu_feiji = (-120, 0, 0)
dandao_qishi = (20000, 0, 2000)
yan_qishi = (17800, 0, 1800)
feiji_qishi = (17800, 0, 1800)

def weizhi_dandao(t):
    return np.array([
        dandao_qishi[0] + t * sudu_dandao[0],
        dandao_qishi[1] + t * sudu_dandao[1],
        dandao_qishi[2] + t * sudu_dandao[2]
    ])

def weizhi_yan(t):
    if t >= 0 and t < 1.5:
        return np.array([
            feiji_qishi[0] + t * sudu_feiji[0],
            feiji_qishi[1] + t * sudu_feiji[1],
            feiji_qishi[2] + t * sudu_feiji[2]
        ])
    if t >= 1.5 and t <= 5.1:
        tziyou = t - 1.5
        z_15 = yan_qishi[2] + 1.5 * sudu_yan[2]
        yan_qishi2 = np.array([
            yan_qishi[0] + 1.5 * sudu_yan[0],
            yan_qishi[1] + 1.5 * sudu_yan[1],
            z_15
        ])
        return np.array([
            yan_qishi2[1] + tziyou * sudu_yan[1],
            yan_qishi2[1] + tziyou * sudu_yan[1],
            yan_qishi2[2] + tziyou * sudu_yan[2] - 0.5 * g * tziyou * tziyou
        ])
    if t > 5.1:
```

```

        tziyou = t - 5.1
        z_51 = yan_qishi[2] + 1.5 * sudu_yan[2] + 3.6 * sudu_yan[2] - 0.5 * g * 3.6 * 3.6
        return np.array([
            yan_qishi[0] + t * sudu_yan2[0],
            yan_qishi[1] + t * sudu_yan2[1],
            z_51 - 3 * tziyou
        ])

def weizhi_yan2(t):
    return (
        yan_qishi[0] + t * sudu_yan2[0],
        yan_qishi[1] + t * sudu_yan2[1],
        yan_qishi[2] + t * sudu_yan2[2]
    )

def weizhi_feiji(t):
    return (
        feiji_qishi[0] + t * sudu_feiji[0],
        feiji_qishi[1] + t * sudu_feiji[1],
        feiji_qishi[2] + t * sudu_feiji[2]
    )

def pan_duanquan(t, eps=1e-12):
    M = weizhi_dandao(t)
    S = np.array(weizhi_yan(t))
    MS = S - M
    juli_MS = np.linalg.norm(MS)
    if juli_MS <= eps:
        return True
    if juli_MS < banjing:
        return True
    sin_a = banjing / juli_MS
    if sin_a >= 1:
        cos_a = 0.0
    else:
        cos_a = math.sqrt(max(0.0, 1.0 - sin_a * sin_a))
    u = MS / juli_MS
    zhu_zhongxin = np.array([zhongxin[0], zhongxin[1], zhongxin[2] + zhu_gaodu / 2.0])
    zhu_fanwei = math.sqrt(zhu_banjing * zhu_banjing + (zhu_gaodu / 2.0) ** 2)
    MC = zhu_zhongxin - M
    juli_MC = np.linalg.norm(MC)
    if juli_MC > 0:
        cos_c = np.dot(MC, u) / (juli_MC)
    else:
        cos_c = 1.0
    if juli_MC * math.cos(math.acos(max(min(cos_c, 1), -1))) >= 0:
        if juli_MC > 0:

```

```

        sin_b = np.linalg.norm(np.cross(MC, u)) / juli_MC
        lhs = juli_MC * sin_b + zhu_fanwei
        rhs = juli_MS * sin_a
        if lhs <= rhs + 1e-9 and np.dot(MC, u) > 0:
            return True
dian_list = [
    np.array([zhongxin[0], zhongxin[1], zhongxin[2] + 0.0]),
    np.array([zhongxin[0], zhongxin[1], zhongxin[2] + zhu_gaodu / 2.0]),
    np.array([zhongxin[0], zhongxin[1], zhongxin[2] + zhu_gaodu])
]
for Pt in dian_list:
    MPt = Pt - M
    norm_MPt = np.linalg.norm(MPt)
    if norm_MPt < eps:
        continue
    cos_t = np.dot(MPt, u) / norm_MPt
    if cos_t < cos_a - 1e-12:
        return False
if abs(u[2]) < 0.999:
    v = np.array([-u[1], u[0], 0.0])
    v_norm = np.linalg.norm(v)
    if v_norm < eps:
        v = np.array([1.0, 0.0, 0.0])
    else:
        v = v / v_norm
else:
    v = np.array([1.0, 0.0, 0.0])
bianjie = []
for z in [zhongxin[2], zhongxin[2] + zhu_gaodu]:
    for sign in [1, -1]:
        Q = np.array([zhongxin[0], zhongxin[1], z]) + sign * zhu_banjing * v
        bianjie.append(Q)
for Q in bianjie:
    MQ = Q - M
    norm_MQ = np.linalg.norm(MQ)
    if norm_MQ < eps:
        continue
    cos_t = np.dot(MQ, u) / norm_MQ
    if cos_t < cos_a - 1e-12:
        return False
if abs(u[2]) > 0.95:
    for z in [zhongxin[2], zhongxin[2] + zhu_gaodu]:
        for k in range(36):
            jiao = 2 * math.pi * k / 36
            Q = np.array([zhongxin[0] + zhu_banjing * math.cos(jiao), zhongxin[1] +
                zhu_banjing * math.sin(jiao), z])
            MQ = Q - M

```

```

        norm_MQ = np.linalg.norm(MQ)
        if norm_MQ < eps:
            continue
        cos_t = np.dot(MQ, u) / norm_MQ
        if cos_t < cos_a - 1e-12:
            return False
    return True

def pan_caiyang(t, n_jiao=64, n_z=8):
    M = weizhi_dandao(t)
    if t >= 1.5 and t <= 5.1:
        S = np.array(weizhi_yan(t))
    elif t >= 0 and t < 1.5:
        S = np.array(weizhi_feiji(t))
    else:
        S = np.array(weizhi_yan2(t))
    MS = S - M
    juli_MS = np.linalg.norm(MS)
    if juli_MS <= 1e-12:
        return True
    sin_a = banjing / juli_MS
    if sin_a >= 1:
        cos_a = 0.0
    else:
        cos_a = math.sqrt(max(0.0, 1.0 - sin_a * sin_a))
    u = MS / juli_MS
    for iz in range(n_z + 1):
        z = zhongxin[2] + (zhu_gaodu) * iz / n_z
        for k in range(n_jiao):
            jiao = 2 * math.pi * k / n_jiao
            Q = np.array([zhongxin[0] + zhu_banjing * math.cos(jiao), zhongxin[1] + zhu_banjing
                          * math.sin(jiao), z])
            MQ = Q - M
            norm_MQ = np.linalg.norm(MQ)
            if norm_MQ < 1e-12:
                continue
            cos_t = np.dot(MQ, u) / norm_MQ
            if cos_t < cos_a - 1e-12:
                return False
    return True

def qujian_list(hanshu, t_max=20.0, dt=0.01):
    t_list = []
    t = 0.0
    while t <= t_max + 1e-12:
        try:
            if hanshu(t):

```

```

        t_list.append(t)
    except Exception as e:
        pass
    t += dt
if not t_list:
    return []
qujian = []
start = t_list[0]
prev = t_list[0]
for tt in t_list[1:]:
    if tt - prev > 1.5 * dt:
        qujian.append((start, prev))
        start = tt
    prev = tt
qujian.append((start, prev))
return qujian

def zongshichang(qujian):
    return sum(t2 - t1 for (t1, t2) in qujian)

if __name__ == "__main__":
    t_max = 20.0
    dt = 0.01
    print("t_max=%.2f, dt=%.3f" % (t_max, dt))
    qujian1 = qujian_list(pan_duanquan, t_max, dt)
    qujian2 = qujian_list(pan_caiyang, t_max, dt)
    shichang1 = zongshichang(qujian1)
    shichang2 = zongshichang(qujian2)
    print("\n解析法区间: ")
    for a,b in qujian1:
        print(" [%.3f, %.3f]" % (a, b))
    print(" 总时长= %.4f s" % shichang1)
    print("\n采样法区间: ")
    for a,b in qujian2:
        print(" [%.3f, %.3f]" % (a, b))
    print(" 总时长= %.4f s" % shichang2)

```

Listing 4: 一个点遮挡成功情形

```
import math
import numpy as np

FY1_weizhi = (17800, 0, 1800)
FY2_weizhi = (12000, 1400, 1400)
FY3_weizhi = (6000, -3000, 700)
FY4_weizhi = (11000, 2000, 1800)
FY5_weizhi = (13000, -2000, 1300)
M1_weizhi = (20000, 0, 2000)
M2_weizhi = (19000, 600, 2100)
M3_weizhi = (18000, -600, 1900)
jiaodu = math.atan(2000/20000)
sudu_M = (-300*math.cos(jiaodu), 0, -300*math.sin(jiaodu))
sudu_FY1 = (-120, 0, 0)
sudu_yanwu = (-120, 0, 0)
sudu_yanwu2 = (0, 0, -3)
FY1_dangqian = (17800, 0, 1800)
M1_dangqian = (20000, 0, 2000)
yanwu_dangqian = (17800, 0, 1800)
shijian1 = 1.5
shijian2 = 5.1
g = 9.8

def weizhi_M1(t):
    x = M1_dangqian[0] + (t) * sudu_M[0]
    y = M1_dangqian[1] + (t) * sudu_M[1]
    z = M1_dangqian[2] + (t) * sudu_M[2]
    return (x, y, z)

def weizhi_yanwu(t):
    if t >= 0 and t < shijian1:
        x = yanwu_dangqian[0] + (t) * sudu_yanwu[0]
        y = yanwu_dangqian[1] + (t) * sudu_yanwu[1]
        z = yanwu_dangqian[2] + (t) * sudu_yanwu[2]
        return (x, y, z)
    elif t >= shijian1 and t <= shijian2:
        x = yanwu_dangqian[0] + shijian1 * sudu_yanwu[0] + (t - shijian1) * sudu_yanwu[0]
        y = yanwu_dangqian[1] + shijian1 * sudu_yanwu[1] + (t - shijian1) * sudu_yanwu[1]
        z = yanwu_dangqian[2] + shijian1 * sudu_yanwu[2] + (t - shijian1) * sudu_yanwu[2] - g *
            (t - shijian1) * (t - shijian1) / 2
        return (x, y, z)
    else:
        x = yanwu_dangqian[0] + shijian1 * sudu_yanwu[0] + 3.6 * sudu_yanwu[0] + (t - shijian2)
            * sudu_yanwu2[0]
        y = yanwu_dangqian[1] + shijian1 * sudu_yanwu[1] + 3.6 * sudu_yanwu[1] + (t - shijian2)
```

```

        * sudu_yanwu2[1]
    z = yanwu_dangqian[2] + shijian1 * sudu_yanwu[2] + 3.6 * sudu_yanwu[2] + (t - shijian2)
        * sudu_yanwu2[2] - g * 3.6 * 3.6 / 2
    return (x, y, z)

P = np.array([0, 200, 0])
qiuti_r = 10

def jiaodian_qiuti(P, Q, C, r):
    PQ = Q - P
    a = np.dot(PQ, PQ)
    b = 2 * np.dot(PQ, P - C)
    c = np.dot(P - C, P - C) - r**2
    D = b*b - 4*a*c
    if D < 0:
        return False
    sqrtD = math.sqrt(D)
    s1 = (-b - sqrtD) / (2*a)
    s2 = (-b + sqrtD) / (2*a)
    return (0 <= s1 <= 1) or (0 <= s2 <= 1)

hits = []
dt = 0.01
T = 25.1

for i in range(int(T/dt) + 1):
    t = round(i * dt, 2)
    Q = np.array(weizhi_M1(t))
    C = np.array(weizhi_yanwu(t))
    if jiaodian_qiuti(P, Q, C, qiuti_r):
        hits.append(t)

if hits:
    t_start = hits[0]
    t_end = hits[-1]
    chixu = round(t_end - t_start, 2)
    print(f"第一次相交时间: {t_start} s")
    print(f"最后一次相交时间: {t_end} s")
    print(f"总持续时间: {chixu} s")
else:
    print("25.1秒内没有发生相交")

```

Listing 5: 圆柱体遮挡成功情形

```

import math
import numpy as np
M1_dangqian_weizhi = (20000, 0, 2000)
yanwu_dangqian_weizhi = (17800, 0, 1800)
jiaodu = math.atan(2000/20000)
sudu_M = (-300*math.cos(jiaodu), 0, -300*math.sin(jiaodu))
sudu_yanwu = (-120, 0, 0)
sudu_yanwu2 = (0, 0, -3)
shijian1 = 1.5
shijian2 = 5.1
g = 9.8
yuanzhu_zhongxin = (0, 200, 0)
yuanzhu_r = 7
yuanzhu_h = 10

def weizhi_M1(t):
    x = M1_dangqian_weizhi[0] + (t) * sudu_M[0]
    y = M1_dangqian_weizhi[1] + (t) * sudu_M[1]
    z = M1_dangqian_weizhi[2] + (t) * sudu_M[2]
    return (x, y, z)

def weizhi_yanwu(t):
    if t >= 0 and t < shijian1:
        x = yanwu_dangqian_weizhi[0] + (t) * sudu_yanwu[0]
        y = yanwu_dangqian_weizhi[1] + (t) * sudu_yanwu[1]
        z = yanwu_dangqian_weizhi[2] + (t) * sudu_yanwu[2]
        return (x, y, z)
    elif t >= shijian1 and t <= shijian2:
        x = yanwu_dangqian_weizhi[0] + shijian1 * sudu_yanwu[0] + (t - shijian1) * sudu_yanwu[0]
        y = yanwu_dangqian_weizhi[1] + shijian1 * sudu_yanwu[1] + (t - shijian1) * sudu_yanwu[1]
        z = yanwu_dangqian_weizhi[2] + shijian1 * sudu_yanwu[2] + (t - shijian1) * sudu_yanwu[2]
        - g * (t - shijian1) * (t - shijian1) / 2
        return (x, y, z)
    else:
        x = yanwu_dangqian_weizhi[0] + shijian1 * sudu_yanwu[0] + 3.6 * sudu_yanwu[0] + (t -
            shijian2) * sudu_yanwu2[0]
        y = yanwu_dangqian_weizhi[1] + shijian1 * sudu_yanwu[1] + 3.6 * sudu_yanwu[1] + (t -
            shijian2) * sudu_yanwu2[1]
        z = yanwu_dangqian_weizhi[2] + shijian1 * sudu_yanwu[2] + 3.6 * sudu_yanwu[2] + (t -
            shijian2) * sudu_yanwu2[2] - g * 3.6 * 3.6 / 2
        return (x, y, z)

def jiaodian_qiuti(P, Q, C, r):
    PQ = Q - P
    a = np.dot(PQ, PQ)

```



```

b = 2 * np.dot(PQ, P - C)
c = np.dot(P - C, P - C) - r**2
D = b*b - 4*a*c
if D < 0:
    return False
sqrtD = math.sqrt(D)
s1 = (-b - sqrtD) / (2*a)
s2 = (-b + sqrtD) / (2*a)
return (0 <= s1 <= 1) or (0 <= s2 <= 1)

def yuanzhu_bianjie_dian(n_theta=64, n_z=4):
    dian = []
    for k in range(n_z+1):
        z = k * yuanzhu_h / n_z
        for i in range(n_theta):
            angle = 2 * math.pi * i / n_theta
            x = yuanzhu_zhongxin[0] + yuanzhu_r * math.cos(angle)
            y = yuanzhu_zhongxin[1] + yuanzhu_r * math.sin(angle)
            dian.append((x, y, z))
    return dian

bianjie_dian = yuanzhu_bianjie_dian()
dt = 0.01
T = 25.1
qiuti_r = 10
youxiao_shijian = []

for i in range(int(T/dt) + 1):
    t = round(i * dt, 2)
    Q = np.array(weizhi_M1(t))
    C = np.array(weizhi_yanwu(t))
    ok = True
    for bp in bianjie_dian:
        P = np.array(bp)
        if not jiaodian_qiuti(P, Q, C, qiuti_r):
            ok = False
            break
    if ok:
        youxiao_shijian.append(t)

if youxiao_shijian:
    t_start = youxiao_shijian[0]
    t_end = youxiao_shijian[-1]
    chixu = round(t_end - t_start, 2)
    print(f"球体完全覆盖圆柱的开始时间: {t_start} s")
    print(f"球体完全覆盖圆柱的结束时间: {t_end} s")
    print(f"总持续时间: {chixu} s")

```

Listing 6: 问题二 GA 遗传算法

```

import math
import numpy as np
from deap import base, creator, tools, algorithms
import random

g = 9.8
r_qiu = 10
p_zhu = np.array([0, 200, 0])
r_zhu = 7
h_zhu = 10
m1_chushi = (20000, 0, 2000)
yan_chushi = (17800, 0, 1800)
jiao = math.atan(2000/20000)
v_m = (-300*math.cos(jiao), 0, -300*math.sin(jiao))

def jisuan_shizhang(v_yan, t1, t2, T=25.1, dt=0.01):
    v_yan2 = (0, 0, -3)
    def weizhi_m1(t):
        return np.array([m1_chushi[0] + t*v_m[0], m1_chushi[1] + t*v_m[1], m1_chushi[2] +
            t*v_m[2]])
    def weizhi_yan(t):
        if t < t1:
            return np.array([yan_chushi[0] + t*v_yan[0], yan_chushi[1] + t*v_yan[1],
                yan_chushi[2] + t*v_yan[2]])
        elif t <= t1 + t2:
            tt = t - t1
            return np.array([yan_chushi[0] + t1*v_yan[0] + tt*v_yan[0], yan_chushi[1] +
                t1*v_yan[1] + tt*v_yan[1], yan_chushi[2] + t1*v_yan[2] + tt*v_yan[2] -
                0.5*g*tt**2])
        else:
            tt = t - (t1 + t2)
            return np.array([yan_chushi[0] + t1*v_yan[0] + t2*v_yan[0] + tt*v_yan2[0],
                yan_chushi[1] + t1*v_yan[1] + t2*v_yan[1] + tt*v_yan2[1], yan_chushi[2] +
                t1*v_yan[2] + t2*v_yan[2] - 0.5*g*t2**2 + tt*v_yan2[2]])
    def chongdie(t, eps=1e-12):
        m = weizhi_m1(t)
        s = weizhi_yan(t)
        ms = s - m
        d_ms = np.linalg.norm(ms)
        if d_ms < eps or d_ms < r_qiu:
            return True
        sin_a = r_qiu / d_ms
        cos_a = math.sqrt(max(0.0, 1 - sin_a**2))
        u = ms / d_ms
        dian = [np.array([p_zhu[0], p_zhu[1], p_zhu[2]]), np.array([p_zhu[0], p_zhu[1], p_zhu[2]

```

```

        + h_zhu/2]), np.array([p_zhu[0], p_zhu[1], p_zhu[2] + h_zhu]))
for p in dian:
    mp = p - m
    d_mp = np.linalg.norm(mp)
    if d_mp < eps: continue
    cos_t = np.dot(mp, u) / d_mp
    if cos_t < cos_a - 1e-12: return False
if abs(u[2]) < 0.999:
    v = np.array([-u[1], u[0], 0.0]); v /= np.linalg.norm(v)
else:
    v = np.array([1,0,0])
for z in [p_zhu[2], p_zhu[2] + h_zhu]:
    for sgn in [1, -1]:
        q = np.array([p_zhu[0], p_zhu[1], z]) + sgn*r_zhu*v
        mq = q - m
        d_mq = np.linalg.norm(mq)
        if d_mq < eps: continue
        cos_t = np.dot(mq, u) / d_mq
        if cos_t < cos_a - 1e-12: return False
    return True
hits = []
for i in range(int(T/dt)+1):
    t = round(i*dt,2)
    if chongdie(t): hits.append(t)
if hits:
    return round(hits[-1]-hits[0],2)
else:
    return 0.0

def zuhua():
    def mubiao(x):
        vx, vy, t1, t2 = x
        return -jisuan_shizhang((vx,vy,0), t1, t2)
    creator.create("FitnessMax", base.Fitness, weights=(1.0,))
    creator.create("Individual", list, fitness=creator.FitnessMax)
    gongju = base.Toolbox()
    def shengcheng():
        while True:
            vx = random.uniform(-200, 0)
            vy = random.uniform(-50, 50)
            norm = math.sqrt(vx**2+vy**2)
            if 70 <= norm <= 140: break
        t1 = random.uniform(0,5)
        t2 = random.uniform(t1,5)
        return creator.Individual([vx,vy,t1,t2])
    gongju.register("individual", shengcheng)
    gongju.register("population", tools.initRepeat, list, gongju.individual)

```

```

gongju.register("evaluate", lambda ind: (-mubiao(ind),))
gongju.register("mate", tools.cxBlend, alpha=0.5)
def bianyi(ind,sigma=5):
    vx,vy,t1,t2=ind
    vx+=random.gauss(0,sigma); vy+=random.gauss(0,sigma)
    norm=math.sqrt(vx**2+vy**2)
    if norm<70: f=70/(norm+1e-8); vx*=f; vy*=f
    elif norm>140: f=140/norm; vx*=f; vy*=f
    t1+=random.gauss(0,sigma/10); t1=max(0,min(t1,5))
    t2+=random.gauss(0,sigma/10); t2=max(t1,min(t2,5))
    ind[:]=[vx,vy,t1,t2]; return ind,
gongju.register("mutate", bianyi)
gongju.register("select", tools.selTournament, tournsize=3)
while True:
    renkou=gongju.population(n=30)
    for gen in range(20):
        haizi=algorithms.varAnd(renkou,gongju,cxpb=0.7,mutpb=0.2)
        fits=list(map(gongju.evaluate,haizi))
        for f,i in zip(fits,haizi): i.fitness.values=f
        renkou=gongju.select(haizi,k=len(renkou))
    best=tools.selBest(renkou,1)[0]
    vx,vy,t1,t2=best
    v_norm=math.sqrt(vx**2+vy**2)
    if 70<=v_norm<=140 and t1>=0 and t2>=t1:
        best_d=-mubiao(best); break
print("GA 最佳参数:",best)
print("GA 最大遮挡时间:",best_d)
return best,best_d

if __name__=="__main__":
    best,best_d=zuhua()

```

Listing 7: 问题二 GA+NM 融合代码

```

import math
import numpy as np
from deap import base, creator, tools, algorithms
from scipy.optimize import minimize
import random

g = 9.8
r_qiu = 10
p_zhu = np.array([0, 200, 0])
r_zhu = 7
h_zhu = 10
m1_chushi = (12000, 0, 1400)
yan_chushi = (11800, 0, 1400)
jiao = math.atan(2000/20000)
v_m = (-300*math.cos(jiao), 0, -300*math.sin(jiao))

def jisuan_zhedang(v_yan, t1, t2, T=25.1, dt=0.01):
    v_yan2 = (0, 0, -3)
    def weizhi_m1(t):
        return np.array([m1_chushi[0]+t*v_m[0], m1_chushi[1]+t*v_m[1], m1_chushi[2]+t*v_m[2]])
    def weizhi_yan(t):
        if t < t1:
            return np.array([yan_chushi[0]+t*v_yan[0], yan_chushi[1]+t*v_yan[1],
                              yan_chushi[2]+t*v_yan[2]])
        elif t <= t1+dt:
            tf = t-t1
            return np.array([yan_chushi[0]+t1*v_yan[0]+tf*v_yan[0],
                              yan_chushi[1]+t1*v_yan[1]+tf*v_yan[1],
                              yan_chushi[2]+t1*v_yan[2]+tf*v_yan[2]-0.5*g*tf**2])
        else:
            tf = t-(t1+dt)
            return np.array([yan_chushi[0]+t1*v_yan[0]+dt*v_yan[0]+tf*v_yan2[0],
                              yan_chushi[1]+t1*v_yan[1]+dt*v_yan[1]+tf*v_yan2[1],
                              yan_chushi[2]+t1*v_yan[2]+dt*v_yan[2]-0.5*g*dt**2+tf*v_yan2[2]])
    def zhu_quanzhe(t, eps=1e-12):
        m = weizhi_m1(t)
        s = weizhi_yan(t)
        ms = s-m
        d = np.linalg.norm(ms)
        if d < eps or d < r_qiu:
            return True
        sin_a = r_qiu/d
        cos_a = math.sqrt(max(0.0, 1-sin_a**2))
        u = ms/d
        dian = [np.array([p_zhu[0], p_zhu[1], p_zhu[2]]),

```

```

        np.array([p_zhu[0],p_zhu[1],p_zhu[2]+h_zhu/2]),
        np.array([p_zhu[0],p_zhu[1],p_zhu[2]+h_zhu]))]
for pt in dian:
    mp = pt-m
    nm = np.linalg.norm(mp)
    if nm < eps: continue
    cos_t = np.dot(mp,u)/nm
    if cos_t < cos_a-1e-12: return False
if abs(u[2])<0.999:
    v = np.array([-u[1],u[0],0.0])
    v/=np.linalg.norm(v)
else:
    v = np.array([1,0,0])
for z in [p_zhu[2],p_zhu[2]+h_zhu]:
    for sgn in [1,-1]:
        q = np.array([p_zhu[0],p_zhu[1],z])+sgn*r_zhu*v
        mq = q-m
        nm = np.linalg.norm(mq)
        if nm < eps: continue
        cos_t = np.dot(mq,u)/nm
        if cos_t < cos_a-1e-12: return False
    return True
hit = []
for i in range(int(T/dt)+1):
    t = round(i*dt,2)
    if zhu_quanzhe(t):
        hit.append(t)
if hit:
    return round(hit[-1]-hit[0],2)
else:
    return 0.0

def youhua():
    def mubiao(x):
        vx,vy,t1,t2=x
        return -jisuan_zhedang((vx,vy,0),t1,t2)
    IND_SIZE=4
    creator.create("Fit",base.Fitness,weights=(1.0,),ignore_type=True)
    creator.create("Ind",list,fitness=creator.Fit)
    gongju=base.Toolbox()
    def shengcheng():
        while True:
            vx=random.uniform(-200,50)
            vy=random.uniform(-200,0)
            nm=math.sqrt(vx**2+vy**2)
            if 70<=nm<=140: break
        t1=random.uniform(0,5)

```

```

        t2=random.uniform(t1,5)
        return creator.Ind([vx,vy,t1,t2])
gongju.register("ind",shengcheng)
gongju.register("pop",tools.initRepeat,list,gongju.ind)
gongju.register("pingjia",lambda ind:(-mubiao(ind),))
gongju.register("jiaopei",tools.cxBlend,alpha=0.5)
def bianyi(ind,sigma=5):
    vx,vy,t1,t2=ind
    vx+=random.gauss(0,sigma)
    vy+=random.gauss(0,sigma)
    nm=math.sqrt(vx**2+vy**2)
    if nm<70: f=70/(nm+1e-8);vx*=f;vy*=f
    elif nm>140: f=140/nm;vx*=f;vy*=f
    t1+=random.gauss(0,sigma/10)
    t1=max(0,min(t1,5))
    t2+=random.gauss(0,sigma/10)
    t2=max(t1,min(t2,5))
    ind[:]=[vx,vy,t1,t2]
    return ind,
gongju.register("bianyi",bianyi)
gongju.register("xuanze",tools.selTournament,tournsize=3)
while True:
    pop=gongju.pop(n=20)
    for gen in range(20):
        zi=algorithms.varAnd(pop,gongju,cxpb=0.7,mutpb=0.2)
        fit=list(map(gongju.pingjia,zi))
        for f,i in zip(fit,zi):
            i.fitness.values=f
        pop=gongju.xuanze(zi,k=len(pop))
        best=tools.selBest(pop,1)[0]
        vx,vy,t1,t2=best
        nm=math.sqrt(vx**2+vy**2)
        if 70<=nm<=140 and t1>=0 and t2>=t1: break
    res=minimize(mubiao,best,method='Nelder-Mead',options={'maxiter':200,'xatol':1e-2,'fatol':1e-2})
    vx,vy,t1,t2=res.x
    nm=math.sqrt(vx**2+vy**2)
    if not(70<=nm<=140 and t1>=0 and t2>=t1):
        return best,-mubiao(best)
    else:
        return res.x,-res.fun

if __name__=="__main__":
    runs=5
    best_t=-1
    best_p=None
    for i in range(runs):
        print(f"第{i+1}次运行:")

```

```
p,t=youhua()
print("本次最大遮挡时间:{:.2f}s\n".format(t))
if t>best_t:
    best_t=t
    best_p=p
print("最佳参数[vx,vy,t1,t2]:",best_p)
print("最大遮挡时间:{:.2f}s".format(best_t))
```


Listing 8: 问题二迭代次数可视化

```
import math
import numpy as np
from deap import base, creator, tools, algorithms
from scipy.optimize import minimize
import random
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

g = 9.8
r_qiu = 10
dian_zhong = np.array([0, 200, 0])
r_yuan = 7
h_yuan = 10

m1_dangqian = (20000, 0, 2000)
yan_dangqian = (17800, 0, 1800)
jiao = math.atan(2000/20000)
su_m = (-300*math.cos(jiao), 0, -300*math.sin(jiao))

def jisuan_zhangai(shu_yan, t1, t2, T=25.1, dt=0.01):
    shu_yan2 = (0, 0, -3)
    def weizhi_m1(t):
        return np.array([m1_dangqian[0] + t*su_m[0],
                        m1_dangqian[1] + t*su_m[1],
                        m1_dangqian[2] + t*su_m[2]])
    def weizhi_yan(t):
        if t < t1:
            return np.array([yan_dangqian[0] + t*shu_yan[0],
                            yan_dangqian[1] + t*shu_yan[1],
                            yan_dangqian[2] + t*shu_yan[2]])
        elif t <= t1 + t2:
            t_ziyou = t - t1
            return np.array([yan_dangqian[0] + t1*shu_yan[0] + t_ziyou*shu_yan[0],
                            yan_dangqian[1] + t1*shu_yan[1] + t_ziyou*shu_yan[1],
                            yan_dangqian[2] + t1*shu_yan[2] + t_ziyou*shu_yan[2] -
                            0.5*g*t_ziyou**2])
        else:
            t_ziyou = t - (t1 + t2)
            return np.array([yan_dangqian[0] + t1*shu_yan[0] + t2*shu_yan[0] +
                            t_ziyou*shu_yan2[0],
                            yan_dangqian[1] + t1*shu_yan[1] + t2*shu_yan[1] + t_ziyou*shu_yan2[1],
                            yan_dangqian[2] + t1*shu_yan[2] + t2*shu_yan[2] - 0.5*g*t2**2 +
                            t_ziyou*shu_yan2[2]])
```

```

def quanfugai(t, eps=1e-12):
    m = weizhi_m1(t)
    s = weizhi_yan(t)
    ms = s - m
    d_ms = np.linalg.norm(ms)
    if d_ms < eps or d_ms < r_qiu:
        return True
    sin_a = r_qiu / d_ms
    cos_a = math.sqrt(max(0.0, 1 - sin_a**2))
    u = ms / d_ms
    dian = [np.array([dian_zhong[0], dian_zhong[1], dian_zhong[2]]),
            np.array([dian_zhong[0], dian_zhong[1], dian_zhong[2]+h_yuan/2]),
            np.array([dian_zhong[0], dian_zhong[1], dian_zhong[2]+h_yuan])]
    for pt in dian:
        mp = pt - m
        d_mp = np.linalg.norm(mp)
        if d_mp < eps: continue
        cos_t = np.dot(mp, u) / d_mp
        if cos_t < cos_a - 1e-12: return False
    if abs(u[2]) < 0.999:
        v = np.array([-u[1], u[0], 0.0])
        v /= np.linalg.norm(v)
    else:
        v = np.array([1,0,0])
    for z in [dian_zhong[2], dian_zhong[2]+h_yuan]:
        for sign in [1, -1]:
            q = np.array([dian_zhong[0], dian_zhong[1], z]) + sign*r_yuan*v
            mq = q - m
            d_mq = np.linalg.norm(mq)
            if d_mq < eps: continue
            cos_t = np.dot(mq, u) / d_mq
            if cos_t < cos_a - 1e-12: return False
    return True

lie = []
for i in range(int(T/dt)+1):
    t = round(i*dt,2)
    if quanfugai(t):
        lie.append(t)
if lie:
    return round(lie[-1] - lie[0],2)
else:
    return 0.0

def youhua():
    def mubiao(x):
        vx, vy, t1, t2 = x
        return jisuan_zhangai((vx, vy, 0), t1, t2)

```

```

if not hasattr(creator, "FitnessMax"):
    creator.create("FitnessMax", base.Fitness, weights=(1.0,), ignore_type=True)
if not hasattr(creator, "Individual"):
    creator.create("Individual", list, fitness=creator.FitnessMax)
gongju = base.Toolbox()
def shengcheng():
    while True:
        vx = random.uniform(-200,0)
        vy = random.uniform(-50,50)
        norm = math.sqrt(vx**2 + vy**2)
        if 70 <= norm <= 140:
            break
    t1 = random.uniform(0,5)
    t2 = random.uniform(t1,5)
    return creator.Individual([vx, vy, t1, t2])
gongju.register("individual", shengcheng)
gongju.register("population", tools.initRepeat, list, gongju.individual)
gongju.register("evaluate", lambda ind: (mubiao(ind),))
gongju.register("mate", tools.cxBlend, alpha=0.5)
def bianyi(ind, sigma=5):
    vx, vy, t1, t2 = ind
    vx += random.gauss(0, sigma)
    vy += random.gauss(0, sigma)
    norm = math.sqrt(vx**2 + vy**2)
    if norm < 70: f = 70/(norm+1e-8); vx*=f; vy*=f
    elif norm > 140: f = 140/norm; vx*=f; vy*=f
    t1 += random.gauss(0, sigma/10)
    t1 = max(0, min(t1,5))
    t2 += random.gauss(0, sigma/10)
    t2 = max(t1, min(t2,5))
    ind[:] = [vx, vy, t1, t2]
    return ind,
gongju.register("mutate", bianyi)
gongju.register("select", tools.selTournament, tournsize=3)
jilu = []
while True:
    renkou = gongju.population(n=30)
    dai = 100
    for gen in range(dai):
        hou = algorithms.varAnd(renkou, gongju, cxpb=0.7, mutpb=0.2)
        fits = list(map(gongju.evaluate, hou))
        for fit, ind in zip(fits, hou):
            ind.fitness.values = fit
        renkou = gongju.select(hou, k=len(renkou))
        best = max([ind.fitness.values[0] for ind in renkou])
        jilu.append(best)
    best_ind = tools.selBest(renkou,1)[0]

```

```

    vx, vy, t1, t2 = best_ind
    norm = math.sqrt(vx**2 + vy**2)
    if 70 <= norm <= 140 and t1>=0 and t2>=t1:
        break
    jieguo = minimize(lambda x: -mubiao(x), best_ind, method='Nelder-Mead',
                      options={'maxiter':200, 'xatol':1e-2, 'fatol':1e-2})
    final_canshu = jieguo.x
    final_shijian = mubiao(jieguo.x)
    plt.figure(figsize=(10,6))
    plt.plot(range(1, len(jilu)+1), jilu, marker='o')
    plt.title("GA迭代过程中总遮挡时间变化", fontsize=16)
    plt.xlabel("迭代次数", fontsize=14)
    plt.ylabel("总遮挡时间 (秒)", fontsize=14)
    plt.grid(True)
    plt.show()
    return final_canshu, final_shijian

if __name__ == "__main__":
    cishu = 1
    best_t = -1
    best_p = None
    for i in range(cishu):
        canshu, shijian = youhua()
        print("本次最大遮挡时间: {:.2f} s\n".format(shijian))
        if shijian > best_t:
            best_t = shijian
            best_p = canshu
    print("最佳参数 [vx, vy, t1, t2]:", best_p)
    print("最大遮挡时间: {:.2f} s".format(best_t))

```

Listing 9: 问题二灵敏度分析

```
# -*- coding: utf-8 -*-
import math
import numpy as np
import os
import csv
import matplotlib.pyplot as plt
from scipy.optimize import minimize
from sklearn.linear_model import LinearRegression
from scipy.stats import pearsonr
import pandas as pd
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

g = 9.8
r = 10
p = np.array([0, 200, 0])
zj_bj = 7
zj_h = 10
m1_dq = (20000, 0, 2000)
yan_dq = (17800, 0, 1800)
jiajiao = math.atan2(2000, 20000)
sd_m = (-300*math.cos(jiajiao), 0.0, -300*math.sin(jiajiao))

def jisuan_zhangai_shijian(sd_yan, t1, t2, T=25.1, dt=0.01):
    sd_yan2 = (0.0, 0.0, -3.0)
    def weizhi_m1(t):
        return np.array([m1_dq[0] + t*sd_m[0],
                        m1_dq[1] + t*sd_m[1],
                        m1_dq[2] + t*sd_m[2]])
    def weizhi_yan(t):
        if t < t1:
            return np.array([yan_dq[0] + t*sd_yan[0],
                            yan_dq[1] + t*sd_yan[1],
                            yan_dq[2] + t*sd_yan[2]])
        elif t <= t1 + t2:
            tf = t - t1
            return np.array([yan_dq[0] + t1*sd_yan[0] + tf*sd_yan[0],
                            yan_dq[1] + t1*sd_yan[1] + tf*sd_yan[1],
                            yan_dq[2] + t1*sd_yan[2] + tf*sd_yan[2] - 0.5*g*tf**2])
        else:
            tf = t - (t1 + t2)
            return np.array([yan_dq[0] + t1*sd_yan[0] + t2*sd_yan[0] + tf*sd_yan2[0],
                            yan_dq[1] + t1*sd_yan[1] + t2*sd_yan[1] + tf*sd_yan2[1],
                            yan_dq[2] + t1*sd_yan[2] + t2*sd_yan[2] - 0.5*g*t2**2 + tf*sd_yan2[2]])
    def shifou_zhangai(t, eps=1e-12):
```

```

m = weizhi_m1(t)
s = weizhi_yan(t)
ms = s - m
jl = np.linalg.norm(ms)
if jl < eps or jl < r:
    return True
sin_jiaodu = r / jl
cos_jiaodu = 0.0 if sin_jiaodu >= 1.0 else math.sqrt(max(0.0, 1 - sin_jiaodu**2))
u = ms / jl
dian_zhou = [np.array([p[0], p[1], p[2]]),
              np.array([p[0], p[1], p[2] + zj_h/2]),
              np.array([p[0], p[1], p[2] + zj_h])]
for pt in dian_zhou:
    mp = pt - m
    jl_mp = np.linalg.norm(mp)
    if jl_mp < eps:
        continue
    cos_t = np.dot(mp, u) / jl_mp
    if cos_t < cos_jiaodu - 1e-12:
        return False
if abs(u[2]) < 0.999:
    v = np.array([-u[1], u[0], 0.0])
    v /= np.linalg.norm(v)
else:
    v = np.array([1.0, 0.0, 0.0])
for z in [p[2], p[2] + zj_h]:
    for fh in [1, -1]:
        q = np.array([p[0], p[1], z]) + fh * zj_bj * v
        mq = q - m
        jl_mq = np.linalg.norm(mq)
        if jl_mq < eps:
            continue
        cos_t = np.dot(mq, u) / jl_mq
        if cos_t < cos_jiaodu - 1e-12:
            return False
    return True
jizhong = []
n = int(T / dt) + 1
for i in range(n):
    t = i * dt
    if shifou_zhangai(t):
        jizhong.append(round(t, 2))
if jizhong:
    ks, js = jizhong[0], jizhong[-1]
    return round(js - ks, 2)
else:
    return 0.0

```

```

def jisuan_params(params):
    vx, vy, t1, t2 = float(params[0]), float(params[1]), float(params[2]), float(params[3])
    return jisuan_zhangai_shijian((vx, vy, 0.0), t1, t2)

def oat_sao(params, fanwei, n=41):
    jieguo = {}
    params = np.array(params, dtype=float)
    for i in range(len(params)):
        low, high = fanwei[i]
        xs = np.linspace(low, high, n)
        ys = []
        for x in xs:
            p1 = params.copy()
            p1[i] = x
            if i == 2 and p1[3] < p1[2]:
                p1[3] = p1[2]
            if i == 3 and p1[3] < p1[2]:
                p1[3] = p1[2]
            ys.append(jisuan_params(p1))
        jieguo[i] = {'x': xs, 'y': np.array(ys)}
    return jieguo

def mc_jubu(params, n=1000, rel_sigma=0.05, time_sigma=0.02, seed=42):
    np.random.seed(seed)
    params = np.array(params, dtype=float)
    X = np.zeros((n, len(params)))
    for i in range(len(params)):
        if i in [0,1]:
            sigma = max(abs(params[i]) * rel_sigma, 1e-3)
            X[:, i] = np.random.normal(loc=params[i], scale=sigma, size=n)
        else:
            sigma = time_sigma
            X[:, i] = np.random.normal(loc=params[i], scale=sigma, size=n)
    for j in range(n):
        if X[j,3] < X[j,2]:
            X[j,3] = X[j,2] + abs(np.random.normal(scale=1e-3))
    y = np.array([jisuan_params(X[j,:]) for j in range(n)])
    return X, y

if __name__ == "__main__":
    os.makedirs("sensitivity_output", exist_ok=True)
    zuijia_canshu = np.array([-77.77482743 ,2.33236065 ,0.63021464, 2.8492512])
    jizhun = jisuan_params(zuijia_canshu)
    print("jizhun =", zuijia_canshu, "shijian =", jizhun)
    names = ['vx', 'vy', 'time1', 'time2']
    fanwei = [

```

```

(zuijia_canshu[0]*0.9, zuijia_canshu[0]*1.1),
(zuijia_canshu[1]*0.9, zuijia_canshu[1]*1.1),
(max(0, zuijia_canshu[2]-1.0), zuijia_canshu[2]+1.0),
(max(zuijia_canshu[2], zuijia_canshu[3]-1.0), zuijia_canshu[3]+1.0)
]
oat_jieguo = oat_sao(zuijia_canshu, fanwei, n=51)
biaoqian = ['vx', 'vy', 't1', 't2']
for i in range(4):
    x = oat_jieguo[i]['x']; y = oat_jieguo[i]['y']
    plt.figure(figsize=(6,4))
    plt.plot(x, y, marker='.', linewidth=1)
    plt.axvline(zuijia_canshu[i], color='k', linestyle='--', label='jizhun')
    plt.title(f"OAT lingminxing fenxi - {biaoqian[i]}", fontsize=12)
    plt.xlabel(biaoqian[i])
    plt.ylabel('shijian (s)')
    plt.grid(True)
    plt.legend()
    plt.tight_layout()
    plt.savefig(f"sensitivity_output/oat_{i+1}_{biaoqian[i]}.png", dpi=150)
    plt.close()

```


Listing 10: 问题三计算求解代码

```

import math
import numpy as np
from deap import base, creator, tools, algorithms
from scipy.optimize import minimize
import random

g = 9.8
R = 10
P = np.array([0, 200, 0])
cyl_r = 7
cyl_h = 10
M1_current_site = (20000, 0, 2000)
smoke_current_site = (17800, 0, 1800)
theta = math.atan(2000/20000)
speed_M = (-300*math.cos(theta), 0, -300*math.sin(theta))

def compute_occlusion_single(speed_smoke, time1, time2, T=25.1, dt=0.01,
    return_interval=False):
    speed_smoke2 = (0, 0, -3)

    def position_M1(t):
        return np.array([M1_current_site[0] + t*speed_M[0],
                        M1_current_site[1] + t*speed_M[1],
                        M1_current_site[2] + t*speed_M[2]])

    def position_smoke(t):
        if t < time1:
            return np.array([smoke_current_site[0] + t*speed_smoke[0],
                            smoke_current_site[1] + t*speed_smoke[1],
                            smoke_current_site[2] + t*speed_smoke[2]])
        elif t <= time1 + time2:
            t_free = t - time1
            return np.array([smoke_current_site[0] + time1*speed_smoke[0] +
                            t_free*speed_smoke[0],
                            smoke_current_site[1] + time1*speed_smoke[1] + t_free*speed_smoke[1],
                            smoke_current_site[2] + time1*speed_smoke[2] + t_free*speed_smoke[2] -
                            0.5*g*t_free**2])
        else:
            t_free = t - (time1 + time2)
            return np.array([smoke_current_site[0] + (time1+time2)*speed_smoke[0] +
                            t_free*speed_smoke2[0],
                            smoke_current_site[1] + (time1+time2)*speed_smoke[1] +
                            t_free*speed_smoke2[1],
                            smoke_current_site[2] + (time1+time2)*speed_smoke[2] - 0.5*g*time2**2

```

```

        + t_free*speed_smoke2[2]])

def is_cylinder_fully_occluded_analytic(t, eps=1e-12):
    M = position_M1(t)
    S = position_smoke(t)
    MS = S - M
    dist_MS = np.linalg.norm(MS)
    if dist_MS < eps or dist_MS < R:
        return True

    sin_alpha = R / dist_MS
    cos_alpha = math.sqrt(max(0.0, 1 - sin_alpha**2))
    u = MS / dist_MS

    axis_points = [np.array([P[0], P[1], P[2]]),
                    np.array([P[0], P[1], P[2]+cyl_h/2]),
                    np.array([P[0], P[1], P[2]+cyl_h])]
    for Pt in axis_points:
        MP = Pt - M
        norm_MP = np.linalg.norm(MP)
        if norm_MP < eps: continue
        cos_theta = np.dot(MP, u) / norm_MP
        if cos_theta < cos_alpha - 1e-12: return False

    if abs(u[2]) < 0.999:
        v = np.array([-u[1], u[0], 0.0])
        v /= np.linalg.norm(v)
    else:
        v = np.array([1,0,0])

    for z in [P[2], P[2]+cyl_h]:
        for sign in [1, -1]:
            Q = np.array([P[0], P[1], z]) + sign*cyl_r*v
            MQ = Q - M
            norm_MQ = np.linalg.norm(MQ)
            if norm_MQ < eps: continue
            cos_theta = np.dot(MQ, u) / norm_MQ
            if cos_theta < cos_alpha - 1e-12: return False
    return True

hits = []
for i in range(int(T/dt)+1):
    t = round(i*dt,2)
    if is_cylinder_fully_occluded_analytic(t):
        hits.append(t)

if hits:

```

```

        t_start, t_end = hits[0], hits[-1]
        duration = round(t_end - t_start, 2)
        if return_interval:
            return duration, t_start, t_end
        return duration
    else:
        if return_interval:
            return 0.0, None, None
        return 0.0

def compute_individual_occlusion(params, T=25.1, dt=0.01):
    vx, vy = params[0], params[1]
    durations = []
    intervals = []
    for i in range(3):
        t1, t2 = params[2+2*i], params[3+2*i]
        dur, ts, te = compute_occlusion_single((vx, vy, 0), t1, t2, T, dt, return_interval=True)
        durations.append(dur)
        if dur > 0:
            intervals.append((i+1, ts, te, dur))
        else:
            intervals.append((i+1, None, None, 0.0))
    return durations, intervals

def compute_total_occlusion_duration(params, T=25.1, dt=0.01):
    vx, vy = params[0], params[1]
    intervals = []
    for i in range(3):
        t1, t2 = params[2+2*i], params[3+2*i]
        dur, ts, te = compute_occlusion_single((vx, vy, 0), t1, t2, T, dt, return_interval=True)
        if dur > 0:
            intervals.append((ts, te))

    if not intervals:
        return 0.0
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
    for s, e in intervals[1:]:
        if s <= merged[-1][1]:
            merged[-1] = (merged[-1][0], max(merged[-1][1], e))
        else:
            merged.append((s, e))
    total = sum(e-s for s, e in merged)
    return total

def optimize_occlusion():
    def objective(x):

```

```

times = [x[2], x[4], x[6]]
times.sort()
penalty = 0
for i in range(1, len(times)):
    if times[i]-times[i-1]<1.0:
        penalty += 1e3
for i in range(3):
    t1, t2 = x[2+2*i], x[3+2*i]
    if t1<=0 or t2<=0 or t2<t1:
        penalty += 1e3
return -compute_total_occlusion_duration(x) + penalty

IND_SIZE = 8
if not hasattr(creator, "FitnessMax"):
    creator.create("FitnessMax", base.Fitness, weights=(1.0,))
if not hasattr(creator, "Individual"):
    creator.create("Individual", list, fitness=creator.FitnessMax)

toolbox = base.Toolbox()

def generate_individual():
    while True:
        vx = random.uniform(-140,-70)
        vy = random.uniform(-70,70)
        norm = math.sqrt(vx**2 + vy**2)
        if not (70 <= norm <= 140):
            continue
        times = []
        for i in range(3):
            t1 = random.uniform(0.1,5)
            t2 = random.uniform(t1,5)
            times.extend([t1, t2])
        ind = [vx, vy] + times
        ts = [ind[2], ind[4], ind[6]]
        ts.sort()
        valid = True
        for i in range(1,3):
            if ts[i]-ts[i-1]<1.0:
                valid = False
                break
        if valid:
            return creator.Individual(ind)

toolbox.register("individual", generate_individual)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
toolbox.register("evaluate", lambda ind: (-objective(ind),))
toolbox.register("mate", tools.cxBlend, alpha=0.5)

```

```

def mutate_individual(ind, sigma=5):
    vx, vy = ind[0]+random.gauss(0,sigma), ind[1]+random.gauss(0,sigma)
    norm = math.sqrt(vx**2+vy**2)
    if norm<70: factor = 70/(norm+1e-8); vx*=factor; vy*=factor
    if norm>140: factor = 140/norm; vx*=factor; vy*=factor
    ind[0], ind[1] = vx, vy
    for i in range(3):
        t1 = ind[2+2*i]+random.gauss(0,sigma/10)
        t2 = ind[3+2*i]+random.gauss(0,sigma/10)
        t1 = max(0.1,min(t1,5))
        t2 = max(t1,min(t2,5))
        ind[2+2*i], ind[3+2*i] = t1, t2
    ts = [ind[2], ind[4], ind[6]]
    ts.sort()
    for i in range(1,3):
        if ts[i]-ts[i-1]<1.0:
            ind[2+2*i] = ts[i-1]+1.0
    return ind,

toolbox.register("mutate", mutate_individual)
toolbox.register("select", tools.selTournament, tournsize=3)

pop = toolbox.population(n=50)
NGEN = 40
for gen in range(NGEN):
    offspring = algorithms.varAnd(pop, toolbox, cxpb=0.7, mutpb=0.3)
    fits = list(map(toolbox.evaluate, offspring))
    for fit, ind in zip(fits, offspring):
        ind.fitness.values = fit
    pop = toolbox.select(offspring, k=len(pop))
best_ind = tools.selBest(pop,1)[0]

res = minimize(objective, best_ind, method='Nelder-Mead',
               options={'maxiter':300, 'xatol':1e-2, 'fatol':1e-2})
return res.x, -res.fun

if __name__ == "__main__":
    runs = 3
    best_overall_duration = -1
    best_overall_params = None

    def check_constraints(params):
        vx, vy = params[0], params[1]
        norm = math.sqrt(vx**2 + vy**2)
        if not (70 <= norm <= 140):

```

```

        return False
    for i in range(3):
        t1, t2 = params[2+2*i], params[3+2*i]
        if t1<=0 or t2<=0 or t2<t1:
            return False
    ts = [params[2], params[4], params[6]]
    ts.sort()
    for i in range(1,3):
        if ts[i]-ts[i-1]<1.0:
            return False
    return True

valid_solution_found = False
max_attempts = 10
attempt = 0

while not valid_solution_found and attempt < max_attempts:
    attempt += 1
    print(f"=== 第 {attempt} 次尝试求解 ===")
    best_overall_duration = -1
    best_overall_params = None

    for i in range(runs):
        print(f"第 {i+1} 次运行优化:")
        params, duration = optimize_occlusion()
        print("本次最大遮挡时间: {:.2f} s".format(duration))
        print("参数 [vx, vy, t1, t2] × 3:", params, "\n")

        if duration > best_overall_duration and check_constraints(params):
            best_overall_duration = duration
            best_overall_params = params

    if best_overall_params is not None:
        valid_solution_found = True

if valid_solution_found:

    print("最佳参数:", best_overall_params)
    print("最大遮挡时间: {:.2f} s".format(best_overall_duration))
    times = [best_overall_params[2], best_overall_params[4], best_overall_params[6]]
    times.sort()
    intervals = [round(times[i]-times[i-1],2) for i in range(1,len(times))]
    print("三枚投放时刻:", [round(t,2) for t in times])
    print("相邻间隔:", intervals)
else:
    print("未找到满足约束的解。")
if valid_solution_found:

```

```

print("最佳参数:", best_overall_params)
print("最大遮挡时间: {:.2f} s".format(best_overall_duration))
times = [best_overall_params[2], best_overall_params[4], best_overall_params[6]]
times.sort()
intervals = [round(times[i]-times[i-1],2) for i in range(1,len(times))]
print("三枚投放时刻:", [round(t,2) for t in times])
print("相邻间隔:", intervals)

# 输出每枚烟幕弹的干扰时长
durations, details = compute_individual_occlusion(best_overall_params)
for idx, (d,info) in enumerate(zip(durations,details), start=1):
    print(f"烟幕弹 {idx}: 干扰时长 = {d:.2f} s, 起始 = {info[1]}, 结束 = {info[2]}")

```

Listing 11: 问题三仿真单独可视化

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.style.use("seaborn-v0_8-whitegrid")
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
plt.rcParams['font.size'] = 12

shuju = [
    {"mingcheng": "干扰弹 1", "kaishi": 4.66, "jieshu": 8.09},
    {"mingcheng": "干扰弹 2", "kaishi": 6.86, "jieshu": 9.95},
    {"mingcheng": "干扰弹 3", "kaishi": 9.93, "jieshu": 11.02},
]

for xiang in shuju:
    xiang["shichang"] = xiang["jieshu"] - xiang["kaishi"]

yanse = sns.color_palette("Set2", len(shuju))

plt.figure(figsize=(8, 2.8))

for i, (xiang, ys) in enumerate(zip(shuju, yanse)):
    plt.barh(i, xiang["shichang"], left=xiang["kaishi"], height=0.45,
             color=ys, edgecolor="black", alpha=0.85, label=xiang["mingcheng"])
    plt.text(xiang["kaishi"] + xiang["shichang"]/2, i, f'{xiang["shichang"]:.2f} s',
             va='center', ha='center', color='black', fontsize=11)

plt.axvline(x=6.86, color='red', linestyle='--', linewidth=1.2, label="重叠区间")
plt.axvline(x=8.09, color='red', linestyle='--', linewidth=1.2)

plt.yticks(range(len(shuju)), [xiang["mingcheng"] for xiang in shuju], fontsize=11)
plt.xlabel("时间 (秒)", fontsize=13, fontweight="bold")
plt.title("干扰弹遮挡时间区间示意图", fontsize=15, fontweight="bold")
plt.xlim(4, 12)

chu, bq = plt.gca().get_legend_handles_labels()
dan = dict(zip(bq, chu))
plt.legend(dan.values(), dan.keys(), loc="upper right", frameon=True)
plt.tight_layout()
plt.show()
```


Listing 12: 问题三格式转换

```

import math
import numpy as np

wurenji_qishi = np.array([17800, 0, 1800])
mudan_qishi = np.array([20000, 0, 2000])
g = 9.8
r = 10
mudan_qishi = (20000, 0, 2000)
yan_qishi = (12000, 1400, 1400)
jiajiao = math.atan(2000/20000)
sudu_mudan = (-300*math.cos(jiajiao), 0, -300*math.sin(jiajiao))
zuijia_canshu = [49.45168072, -41.60111993, 2.45303911, 5.2137012]

vx, vy = zuijia_canshu[0], zuijia_canshu[1]
sudu = math.sqrt(vx**2 + vy**2)
jiaodu_rad = math.atan2(vy, vx)
jiaodu_du = math.degrees(jiaodu_rad)
if jiaodu_du < 0:
    jiaodu_du += 360
jiaodu_du = round(jiaodu_du, 4)
jiaodu_du = jiaodu_du % 360
sudu = round(sudu, 4)

sudu_yan = (vx, vy, 0)
sudu_yan2 = (0, 0, -3)
jiegua = []
for i in range(1):
    t1, t2 = zuijia_canshu[2+2*i], zuijia_canshu[3+2*i]

    def weizhi_yan(t):
        if t >= 0 and t < t1:
            x = yan_qishi[0] + t * sudu_yan[0]
            y = yan_qishi[1] + t * sudu_yan[1]
            z = yan_qishi[2] + t * sudu_yan[2]
            return (x, y, z)
        elif t >= t1 and t <= (t1 + t2):
            x = yan_qishi[0] + t1 * sudu_yan[0] + (t - t1) * sudu_yan[0]
            y = yan_qishi[1] + t1 * sudu_yan[1] + (t - t1) * sudu_yan[1]
            z = yan_qishi[2] + t1 * sudu_yan[2] + (t - t1) * sudu_yan[2] - g * (t - t1) * (t - t1) / 2
            return (x, y, z)
        else:
            x = yan_qishi[0] + t1 * sudu_yan[0] + t2 * sudu_yan[0] + (t - (t1 + t2)) * sudu_yan2[0]
            y = yan_qishi[1] + t1 * sudu_yan[1] + t2 * sudu_yan[1] + (t - (t1 + t2)) *

```

```

        sudu_yan2[1]
        z = yan_qishi[2] + t1 * sudu_yan[2] + t2 * sudu_yan[2] + (t - (t1 + t2)) *
            sudu_yan2[2] - g * t2 * t2 / 2
        return (x, y, z)

x0 = weizhi_yan(t1)[0]
y0 = weizhi_yan(t1)[1]
z0 = weizhi_yan(t1)[2]
x_qibao = weizhi_yan(t2)[0]
y_qibao = weizhi_yan(t2)[1]
z_qibao = weizhi_yan(t2)[2]
youxiao_shichang = round(t2, 4)

jieguo.append({
    "yundong_fangxiang": jiaodu_du,
    "yundong_sudu": sudu,
    "bianhao": i+1,
    "toufang_x": round(x0, 4),
    "toufang_y": round(y0, 4),
    "toufang_z": round(z0, 4),
    "qibao_x": round(x_qibao, 4),
    "qibao_y": round(y_qibao, 4),
    "qibao_z": round(z_qibao, 4),
    "youxiao_shichang": youxiao_shichang
})

for hang in jieguo:
    print(hang)

x_mubiao = 10876.1634
x_qishi = yan_qishi[0]
y_qishi = yan_qishi[1]
vx = zuijia_canshu[0]
vy = zuijia_canshu[1]

t_duiying = (x_mubiao - x_qishi) / vx
y_mubiao = y_qishi + vy * t_duiying

print("对应y坐标:", y_mubiao)

```

Listing 13: 问题四计算求解代码

```

import math
import numpy as np
from deap import base, creator, tools, algorithms
from scipy.optimize import minimize
import random

g = 9.8
R = 10
P = np.array([0, 200, 0])
cyl_r = 7
cyl_h = 10
M1_current_site = (20000, 0, 2000)
theta = math.atan(2000/20000)
speed_M = (-300*math.cos(theta), 0, -300*math.sin(theta))
def compute_occlusion_duration_with_intervals(speed_smoke, smoke_current_site, time1, time2,
    T=66, dt=0.01):
    speed_smoke2 = (0, 0, -3)
    def position_M1(t):
        return np.array([M1_current_site[0] + t*speed_M[0],
            M1_current_site[1] + t*speed_M[1],
            M1_current_site[2] + t*speed_M[2]])
    def position_smoke(t):
        if t < time1:
            return np.array([smoke_current_site[0] + t*speed_smoke[0],
                smoke_current_site[1] + t*speed_smoke[1],
                smoke_current_site[2] + t*speed_smoke[2]])
        elif t <= time1 + time2:
            t_free = t - time1
            return np.array([smoke_current_site[0] + time1*speed_smoke[0] +
                t_free*speed_smoke[0],
                smoke_current_site[1] + time1*speed_smoke[1] + t_free*speed_smoke[1],
                smoke_current_site[2] + time1*speed_smoke[2] + t_free*speed_smoke[2] -
                0.5*g*t_free**2])
        else:
            t_free = t - (time1 + time2)
            return np.array([smoke_current_site[0] + (time1+time2)*speed_smoke[0] +
                t_free*speed_smoke2[0],
                smoke_current_site[1] + (time1+time2)*speed_smoke[1] +
                t_free*speed_smoke2[1],
                smoke_current_site[2] + (time1+time2)*speed_smoke[2] - 0.5*g*time2**2
                + t_free*speed_smoke2[2]])

    def is_cylinder_fully_occluded_analytic(t, eps=1e-12):
        M = position_M1(t)

```

```

S = position_smoke(t)
MS = S - M
dist_MS = np.linalg.norm(MS)
if dist_MS < eps or dist_MS < R:
    return True
sin_alpha = R / dist_MS
cos_alpha = math.sqrt(max(0.0, 1 - sin_alpha**2))
u = MS / dist_MS

axis_points = [np.array([P[0], P[1], P[2]]),
                np.array([P[0], P[1], P[2]+cyl_h/2]),
                np.array([P[0], P[1], P[2]+cyl_h])]
for Pt in axis_points:
    MP = Pt - M
    norm_MP = np.linalg.norm(MP)
    if norm_MP < eps: continue
    cos_theta = np.dot(MP, u) / norm_MP
    if cos_theta < cos_alpha - 1e-12: return False

if abs(u[2]) < 0.999:
    v = np.array([-u[1], u[0], 0.0])
    v /= np.linalg.norm(v)
else:
    v = np.array([1,0,0])

for z in [P[2], P[2]+cyl_h]:
    for sign in [1, -1]:
        Q = np.array([P[0], P[1], z]) + sign*cyl_r*v
        MQ = Q - M
        norm_MQ = np.linalg.norm(MQ)
        if norm_MQ < eps: continue
        cos_theta = np.dot(MQ, u) / norm_MQ
        if cos_theta < cos_alpha - 1e-12: return False
    return True

hits = []
for i in range(int(T/dt)+1):
    t = round(i*dt,2)
    if is_cylinder_fully_occluded_analytic(t):
        hits.append(t)

intervals = []
if hits:
    start = hits[0]
    for i in range(1, len(hits)):
        if round(hits[i] - hits[i-1], 2) > dt:
            end = hits[i-1]

```

```

        intervals.append((start, end))
        start = hits[i]
        intervals.append((start, hits[-1]))
        total_duration = sum([end - start for start, end in intervals])
        return total_duration, intervals
    else:
        return 0.0, []

def solve_smoke_strategy(smoke_current_site, runs=5):
    def objective(x):
        vx, vy, time1, time2 = x
        duration, _ = compute_occlusion_duration_with_intervals((vx, vy, 0), smoke_current_site,
                                                                time1, time2)
        return -duration

    if not hasattr(creator, "FitnessMax"):
        creator.create("FitnessMax", base.Fitness, weights=(1.0,))
    if not hasattr(creator, "Individual"):
        creator.create("Individual", list, fitness=creator.FitnessMax)

    toolbox = base.Toolbox()

    def generate_individual():
        while True:
            vx = random.uniform(-50,140)
            vy = random.uniform(0,200)
            norm = math.sqrt(vx**2 + vy**2)
            if 70 <= norm <= 140:
                break
            time1 = random.uniform(46,66)
            time2 = random.uniform(time1,66)
            return creator.Individual([vx, vy, time1, time2])

    toolbox.register("individual", generate_individual)
    toolbox.register("population", tools.initRepeat, list, toolbox.individual)
    toolbox.register("evaluate", lambda ind: (-objective(ind),))
    toolbox.register("mate", tools.cxBlend, alpha=0.5)

    def mutate_individual(ind, sigma=5):
        vx, vy, time1, time2 = ind
        vx += random.gauss(0, sigma)
        vy += random.gauss(0, sigma)
        norm = math.sqrt(vx**2 + vy**2)
        if norm < 70: factor = 70/(norm+1e-8); vx*=factor; vy*=factor
        elif norm > 140: factor = 140/norm; vx*=factor; vy*=factor
        time1 += random.gauss(0, sigma/10)
        time1 = max(0, min(time1,66))

```

```

time2 += random.gauss(0, sigma/10)
time2 = max(time1, min(time2,66))
ind[:] = [vx, vy, time1, time2]
return ind,

toolbox.register("mutate", mutate_individual)
toolbox.register("select", tools.selTournament, tournsize=3)

best_overall_duration = -1
best_overall_params = None
best_overall_intervals = []

for i in range(runs):
    print(f"\n第 {i+1} 次运行优化:")
    while True:
        pop = toolbox.population(n=40)
        NGEN = 15
        for gen in range(NGEN):
            offspring = algorithms.varAnd(pop, toolbox, cxpb=0.7, mutpb=0.2)
            fits = list(map(toolbox.evaluate, offspring))
            for fit, ind in zip(fits, offspring):
                ind.fitness.values = fit
            pop = toolbox.select(offspring, k=len(pop))
        best_ind = tools.selBest(pop,1)[0]
        vx, vy, t1, t2 = best_ind
        speed_norm = math.sqrt(vx**2 + vy**2)
        if 70 <= speed_norm <= 140 and t1>=0 and t2>=t1:
            break

    res = minimize(objective, best_ind, method='Nelder-Mead',
                   options={'maxiter':200, 'xatol':1e-2, 'fatol':1e-2})
    vx, vy, t1, t2 = res.x
    duration, intervals = compute_occlusion_duration_with_intervals((vx, vy, 0),
                          smoke_current_site, t1, t2)
    print("运行 {} 参数: vx={:.4f}, vy={:.4f}, time1={:.4f}, time2={:.4f}".format(i+1, vx,
                                          vy, t1, t2))
    print("本次最大遮挡时间: {:.4f} s".format(duration))
    print("遮蔽时间区间:", intervals)
    if duration > best_overall_duration:
        best_overall_duration = duration
        best_overall_params = res.x
        best_overall_intervals = intervals

print("最佳参数 [vx, vy, time1, time2]:", best_overall_params)
print("最大遮挡时间: {:.4f} s".format(best_overall_duration))
print("遮蔽时间区间:", best_overall_intervals)
return best_overall_params, best_overall_duration, best_overall_intervals

```

```
if __name__ == "__main__":  
    smoke_site = (6000, -3000, 700)  
    solve_smoke_strategy(smoke_site)
```

Listing 14: 问题五各自求解

```
import math
import numpy as np
from deap import base, creator, tools, algorithms
from scipy.optimize import minimize
import random

g = 9.8
R = 10
P = np.array([0, 200, 0])
cyl_r = 7
cyl_h = 10

missiles = {
    "M1": (20000, 0, 2000),
    "M2": (19000, 600, 2100),
    "M3": (18000, -600, 1900)
}

uavs = {
    "FY1": (17800, 0, 1800),
    "FY2": (12000, 1400, 1400),
    "FY3": (6000, -3000, 700),
    "FY4": (11000, 2000, 1800),
    "FY5": (13000, -2000, 1300)
}

assignment = {
    "M1": ["FY1"],
    "M2": ["FY2", "FY4", "FY5"],
    "M3": ["FY3"]
}

def compute_missile_speed(M_current_site, speed=300):
    dx = P[0] - M_current_site[0]
    dy = P[1] - M_current_site[1]
    dz = P[2] - M_current_site[2]
    dist = math.sqrt(dx**2 + dy**2 + dz**2)
    return (speed*dx/dist, speed*dy/dist, speed*dz/dist)

def compute_occlusion_single(M_current_site, speed_M, smoke_current_site,
                             speed_smoke, time1, time2,
                             T=25.1, dt=0.01, return_interval=False):
    speed_smoke2 = (0, 0, -3)
```



```

def position_M(t):
    return np.array([M_current_site[0] + t*speed_M[0],
                     M_current_site[1] + t*speed_M[1],
                     M_current_site[2] + t*speed_M[2]])

def position_smoke(t):
    if t < time1:
        return np.array([smoke_current_site[0] + t*speed_smoke[0],
                         smoke_current_site[1] + t*speed_smoke[1],
                         smoke_current_site[2] + t*speed_smoke[2]])
    elif t <= time1 + time2:
        t_free = t - time1
        return np.array([smoke_current_site[0] + time1*speed_smoke[0] +
                         t_free*speed_smoke[0],
                         smoke_current_site[1] + time1*speed_smoke[1] + t_free*speed_smoke[1],
                         smoke_current_site[2] + time1*speed_smoke[2] + t_free*speed_smoke[2] -
                         0.5*g*t_free**2])
    else:
        t_free = t - (time1 + time2)
        return np.array([smoke_current_site[0] + (time1+time2)*speed_smoke[0] +
                         t_free*speed_smoke2[0],
                         smoke_current_site[1] + (time1+time2)*speed_smoke[1] +
                         t_free*speed_smoke2[1],
                         smoke_current_site[2] + (time1+time2)*speed_smoke[2] - 0.5*g*time2**2
                         + t_free*speed_smoke2[2]])

def is_cylinder_fully_occluded_analytic(t, eps=1e-12):
    M = position_M(t)
    S = position_smoke(t)
    MS = S - M
    dist_MS = np.linalg.norm(MS)
    if dist_MS < eps or dist_MS < R:
        return True

    sin_alpha = R / dist_MS
    cos_alpha = math.sqrt(max(0.0, 1 - sin_alpha**2))
    u = MS / dist_MS

    axis_points = [np.array([P[0], P[1], P[2]]),
                   np.array([P[0], P[1], P[2]+cyl_h/2]),
                   np.array([P[0], P[1], P[2]+cyl_h])]
    for Pt in axis_points:
        MP = Pt - M
        norm_MP = np.linalg.norm(MP)
        if norm_MP < eps: continue
        cos_theta = np.dot(MP, u) / norm_MP
        if cos_theta < cos_alpha - 1e-12: return False

```

```

    if abs(u[2]) < 0.999:
        v = np.array([-u[1], u[0], 0.0])
        v /= np.linalg.norm(v)
    else:
        v = np.array([1,0,0])

    for z in [P[2], P[2]+cyl_h]:
        for sign in [1, -1]:
            Q = np.array([P[0], P[1], z]) + sign*cyl_r*v
            MQ = Q - M
            norm_MQ = np.linalg.norm(MQ)
            if norm_MQ < eps: continue
            cos_theta = np.dot(MQ, u) / norm_MQ
            if cos_theta < cos_alpha - 1e-12: return False
        return True

hits = []
for i in range(int(T/dt)+1):
    t = round(i*dt,2)
    if is_cylinder_fully_occluded_analytic(t):
        hits.append(t)

if hits:
    t_start, t_end = hits[0], hits[-1]
    duration = round(t_end - t_start,2)
    if return_interval:
        return duration, t_start, t_end
    return duration
else:
    if return_interval:
        return 0.0, None, None
    return 0.0

def compute_individual_occlusion(params, M_current_site, speed_M, smoke_current_site, T=50,
    dt=0.01):
    vx, vy = params[0], params[1]
    durations = []
    intervals = []
    for i in range(3):
        t1, t2 = params[2+2*i], params[3+2*i]
        dur, ts, te = compute_occlusion_single(M_current_site, speed_M, smoke_current_site,
            (vx, vy, 0), t1, t2, T, dt, return_interval=True)
        durations.append(dur)
        intervals.append((i+1, ts, te, dur))
    return durations, intervals

```

```

def compute_total_occlusion_duration(params, M_current_site, speed_M, smoke_current_site,
    T=50, dt=0.01):
    vx, vy = params[0], params[1]
    intervals = []
    for i in range(3):
        t1, t2 = params[2+2*i], params[3+2*i]
        dur, ts, te = compute_occlusion_single(M_current_site, speed_M, smoke_current_site,
            (vx, vy, 0), t1, t2, T, dt, return_interval=True)

        if dur > 0:
            intervals.append((ts, te))
    if not intervals:
        return 0.0
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
    for s, e in intervals[1:]:
        if s <= merged[-1][1]:
            merged[-1] = (merged[-1][0], max(merged[-1][1], e))
        else:
            merged.append((s, e))
    return sum(e-s for s, e in merged)

def optimize_occlusion(M_current_site, speed_M, smoke_current_site):
    def objective(x):
        times = [x[2], x[4], x[6]]
        times.sort()
        penalty = 0
        for i in range(1, len(times)):
            if times[i]-times[i-1]<1.0:
                penalty += 1e3
        for i in range(3):
            t1, t2 = x[2+2*i], x[3+2*i]
            if t1<=0 or t2<=0 or t2<t1:
                penalty += 1e3
        return -compute_total_occlusion_duration(x, M_current_site, speed_M, smoke_current_site)
            + penalty

    IND_SIZE = 8
    if not hasattr(creator, "FitnessMax"):
        creator.create("FitnessMax", base.Fitness, weights=(1.0,))
    if not hasattr(creator, "Individual"):
        creator.create("Individual", list, fitness=creator.FitnessMax)

    toolbox = base.Toolbox()
    def generate_individual():
        while True:
            vx = random.uniform(-140,-140)
            vy = random.uniform(-140,140)

```

```

    norm = math.sqrt(vx**2 + vy**2)
    if not (70 <= norm <= 140):
        continue
    times = []
    for i in range(3):
        t1 = random.uniform(25,50)
        t2 = random.uniform(t1,50)
        times.extend([t1, t2])
    ind = [vx, vy] + times
    ts = [ind[2], ind[4], ind[6]]
    ts.sort()
    valid = True
    for i in range(1,3):
        if ts[i]-ts[i-1]<1.0:
            valid = False
            break
    if valid:
        return creator.Individual(ind)

toolbox.register("individual", generate_individual)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
toolbox.register("evaluate", lambda ind: (-objective(ind),))
toolbox.register("mate", tools.cxBlend, alpha=0.5)

def mutate_individual(ind, sigma=5):
    vx, vy = ind[0]+random.gauss(0,sigma), ind[1]+random.gauss(0,sigma)
    norm = math.sqrt(vx**2+vy**2)
    if norm<70: factor = 70/(norm+1e-8); vx*=factor; vy*=factor
    if norm>140: factor = 140/norm; vx*=factor; vy*=factor
    ind[0], ind[1] = vx, vy
    for i in range(3):
        t1 = ind[2+2*i]+random.gauss(0,sigma/10)
        t2 = ind[3+2*i]+random.gauss(0,sigma/10)
        t1 = max(0.1,min(t1,5))
        t2 = max(t1,min(t2,5))
        ind[2+2*i], ind[3+2*i] = t1, t2
    ts = [ind[2], ind[4], ind[6]]
    ts.sort()
    for i in range(1,3):
        if ts[i]-ts[i-1]<1.0:
            ind[2+2*i] = ts[i-1]+1.0
    return ind,

toolbox.register("mutate", mutate_individual)
toolbox.register("select", tools.selTournament, tournsize=3)

pop = toolbox.population(n=50)

```

```

NGEN = 40
for gen in range(NGEN):
    offspring = algorithms.varAnd(pop, toolbox, cxpb=0.7, mutpb=0.3)
    fits = list(map(toolbox.evaluate, offspring))
    for fit, ind in zip(fits, offspring):
        ind.fitness.values = fit
    pop = toolbox.select(offspring, k=len(pop))
    best_ind = tools.selBest(pop,1)[0]

    res = minimize(objective, best_ind, method='Nelder-Mead',
                   options={'maxiter':300, 'xatol':1e-2, 'fatol':1e-2})
    return res.x, -res.fun

if __name__ == "__main__":
    results = {}
    for missile, uav_list in assignment.items():
        M_site = np.array(missiles[missile])
        speed_M = compute_missile_speed(M_site)

        min_dist, chosen_uav = float("inf"), None
        for uav in uav_list:
            d = np.linalg.norm(np.array(uavs[uav]) - M_site)
            if d < min_dist:
                min_dist, chosen_uav = d, uav
        smoke_site = uavs[chosen_uav]

        print(f"\n=== 优化 {missile} (执行无人机: {chosen_uav}) ===")
        params, duration = optimize_occlusion(M_site, speed_M, smoke_site)
        durations, details = compute_individual_occlusion(params, M_site, speed_M, smoke_site)

        results[missile] = {
            "uav": chosen_uav,
            "params": params,
            "total_duration": duration,
            "details": details
        }

    for m, info in results.items():
        print(f"{m}: 执行无人机={info['uav']}, 遮挡时间={info['total_duration']:.2f}s")
        print(f" 参数={info['params']}")
        for det in info["details"]:
            print(f" 烟幕弹{det[0]}: 时长={det[3]:.2f}s, 起始={det[1]}, 结束={det[2]}")

```

Listing 15: 问题五聚类求解

```
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

daodan = {
    "M1": (20000, 0, 2000),
    "M2": (19000, 600, 2100),
    "M3": (18000, -600, 1900)
}

wurenji = {
    "FY1": (17800, 0, 1800),
    "FY2": (12000, 1400, 1400),
    "FY3": (6000, -3000, 700),
    "FY4": (11000, 2000, 1800),
    "FY5": (13000, -2000, 1300)
}

wurenji_mingzi = list(wurenji.keys())
wurenji_zuobiao = np.array(list(wurenji.values()))

kmeans = KMeans(n_clusters=len(daodan), random_state=42)
biaoqian = kmeans.fit_predict(wurenji_zuobiao)

for i, mingzi in enumerate(wurenji_mingzi):
    print(f"{mingzi} -> qunzu {biaoqian[i]}")

for i, zhongxin in enumerate(kmeans.cluster_centers_):
    print(f"qunzu {i}: {zhongxin}")

tu1 = plt.figure(figsize=(10, 7))
ax = tu1.add_subplot(111, projection='3d')

ax.scatter(wurenji_zuobiao[:, 0], wurenji_zuobiao[:, 1], wurenji_zuobiao[:, 2],
           c=biaoqian, cmap='tab10', s=80, label="wurenji")

for i, mingzi in enumerate(wurenji_mingzi):
    ax.text(wurenji_zuobiao[i, 0], wurenji_zuobiao[i, 1], wurenji_zuobiao[i, 2],
           mingzi, fontsize=10, color="black")

daodan_zuobiao = np.array(list(daodan.values()))
ax.scatter(daodan_zuobiao[:, 0], daodan_zuobiao[:, 1], daodan_zuobiao[:, 2],
           c="red", marker="^", s=150, label="daodan")
```

```

for i, mingzi in enumerate(daodan.keys()):
    ax.text(daodan_zuobiao[i, 0], daodan_zuobiao[i, 1], daodan_zuobiao[i, 2],
            mingzi, fontsize=12, color="red")

zhongxin = kmeans.cluster_centers_
ax.scatter(zhongxin[:, 0], zhongxin[:, 1], zhongxin[:, 2],
           c="blue", marker="*", s=200, label="qunzu zhongxin")

ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_zlabel("Z")
ax.set_title("wurenji-daodan fenpei (K-Means qunzu)")
ax.legend()
plt.show()

tu2, ax2 = plt.subplots(figsize=(10, 7))

ax2.scatter(wurenji_zuobiao[:, 0], wurenji_zuobiao[:, 1],
            c=biaoqian, cmap='tab10', s=80, label="wurenji")

for i, mingzi in enumerate(wurenji_mingzi):
    ax2.text(wurenji_zuobiao[i, 0], wurenji_zuobiao[i, 1],
            mingzi, fontsize=10, color="black")

ax2.scatter(daodan_zuobiao[:, 0], daodan_zuobiao[:, 1],
            c="red", marker="^", s=150, label="daodan")

for i, mingzi in enumerate(daodan.keys()):
    ax2.text(daodan_zuobiao[i, 0], daodan_zuobiao[i, 1],
            mingzi, fontsize=12, color="red")

ax2.scatter(zhongxin[:, 0], zhongxin[:, 1],
            c="blue", marker="*", s=200, label="qunzu zhongxin")

ax2.set_xlabel("X")
ax2.set_ylabel("Y")
ax2.set_title("wurenji-daodan fenpei (XY pingmian touying)")
ax2.legend()
ax2.grid(True)
plt.show()

```